Diffusion Models: U-Net vs DiT, and How Conditioning Really Works

- Diffusion Models: U-Net vs DiT, and How Conditioning *Really* Works
  - Table of contents
- Part I — Architecture and scaling
  - 1. Adding a U-Net stage: receptive field and minimum image size
  - 2. Wait — attention *inside* a U-Net?
  - 3. Does “bigger = better” actually hold for DiT? What knobs do I have?
  - 4. The four kinds of parallelism, and why uniform blocks matter
  - 5. Mixture-of-Experts: code, line by line, train vs inference
  - 6. RoPE, NaViT packing, and the aspect-ratio / video-length problem
  - 7. MMDiT in code, row by row
  - 8. Why a U-Net can *only* bolt text on from the side
- Part II — Choosing a backbone in practice
  - 9. From scratch vs fine-tune: U-Net or DiT?
  - 10. How much data do I actually need?
  - 11. The “U-Net prior” and spatially-local tasks
  - 12. Flow matching caveats: timestep sampling and the resolution shift
  - 13. LoRA for diffusion models
- Part III — Conditioning, re-explained from zero
  - 14. The four ways to inject the timestep. adaLN vs adaLN-Zero
  - 15. One vector or many? The canonical conditioning recipe
  - 16. The red-cube / blue-sphere argument, slowly
  - 17. What SDXL, SD3, Flux actually are (a family tree)
  - 18. CLIP pooled *and* CLIP sequence — why both
  - 19. Why not cross-attend to a depth map? The identity-permutation argument
  - 20. Input concat, ControlNet, token concat — a worked inpainting example
- Part IV
  - 21. Learnings to remember (the compressed version)
  - 22. A prompt for another Claude terminal

Diffusion Models: U-Net vs DiT, and How Conditioning *Really* Works

A long-form answer document. Every question you asked, answered from first principles, with code and shape traces.

Table of contents

Part I — Architecture and scaling 1. Adding a U-Net stage: receptive field and minimum image size 2. Wait — attention *inside* a U-Net? 3. Does "bigger = better" actually hold for DiT? What knobs do I have? 4. The four kinds of parallelism, and why uniform blocks matter 5. Mixture-of-Experts: code, line by line, train vs inference 6. RoPE, NaViT packing, and the aspect-ratio / video-length problem 7. MMDiT in code, row by row 8. Why a U-Net can *only* bolt text on from the side

Part II — Choosing a backbone in practice 9. From scratch vs fine-tune: U-Net or DiT? 10. How much data do I actually need? (images and video) 11. The "U-Net prior" and spatially-local tasks 12. Flow matching caveats: timestep sampling and the resolution shift 13. LoRA for diffusion models

Part III — Conditioning, re-explained from zero 14. The four ways to inject the timestep. adaLN vs adaLN-Zero 15. One vector or many? The canonical conditioning recipe 16. The red-cube / blue-sphere argument, slowly 17. What SDXL, SD3, Flux actually are (a family tree) 18. CLIP pooled *and* CLIP sequence — why both 19. Why not cross-attend to a depth map? The identity-permutation argument 20. Input concat, ControlNet, token concat — a worked inpainting example

Part IV 21. Learnings to remember (the compressed version) 22. A prompt for another Claude terminal to build the teaching files

Part I — Architecture and scaling

1. Adding a U-Net stage: receptive field and minimum image size

The receptive field gets bigger

Here is the mechanism, precisely. A U-Net stage operates at some resolution. Each stage ends with a `stride=2` downsample, so stage *k* runs at resolution `H / 2^k`.

Now: a `3×3` convolution always looks at a `3×3` neighbourhood **of its own input grid**. But its own input grid is downsampled. So in terms of *original image pixels*, a `3×3` conv at stage *k* covers `3 × 2^k` pixels per side:

```
stage 0   resolution 64×64   a 3×3 conv spans   3 original pixels
stage 1   resolution 32×32   a 3×3 conv spans   6 original pixels
stage 2   resolution 16×16   a 3×3 conv spans  12 original pixels
stage 3   resolution  8×8    a 3×3 conv spans  24 original pixels
stage 4   resolution  4×4    a 3×3 conv spans  48 original pixels
```

So **adding a deeper stage enlarges the receptive field** — it is the *only* cheap way a pure-conv network can see globally. This is the entire reason U-Nets are shaped like a U: you can't afford global context at full resolution, so you shrink the image until "3×3" means "most of the picture", do your global reasoning there, then upsample back and use the skip connections to restore the fine detail you threw away.

A useful way to hold it: **downsampling is how a CNN buys global receptive field with local operators.** Attention buys it directly, at `O(N²)`.

The minimum image size gets bigger

Two separate constraints:

(a) Divisibility. If the network downsamples by 2 a total of `D` times, every downsample must divide evenly, so the input side length must be a multiple of `2^D`. Concretely for Stable Diffusion:

```
image 512×512
  → VAE encoder downsamples ×8      → latent 64×64
  → U-Net has 3 downsamples (×8)    → bottleneck 8×8

so image side must be divisible by 8 (VAE) × 8 (U-Net) = 64 pixels.
```

That is exactly why every SD UI forces you to pick dimensions in multiples of 64. Add one more U-Net stage → `2^4 = 16` → images must be multiples of 128.

(b) The bottleneck must still be spatially meaningful. A `3×3` conv on a `2×2` feature map is nearly all padding; a self-attention over 4 tokens can barely express anything. In practice you want the bottleneck ≥ `4×4`, preferably `8×8`. So on a `64×64` latent you get at most 4 downsamples, and 3 is the standard choice.

Why this is a *scaling* problem, not just a trivia point

Look at what one architectural decision — "add a stage" — simultaneously changes:

You add a stage and…	consequence
receptive field	grows (good)
minimum input size	doubles (constrains your dataset and your product)
bottleneck resolution	halves (may become too coarse to be useful)
parameter count	grows, but only in the *deepest, cheapest* part of the net
where attention is affordable	shifts (a new, even cheaper resolution appears)
skip-connection topology	changes (a new encoder↔decoder pair)

Six coupled effects from one knob. That is what "hand-designed ladder" means. Contrast a DiT: increment `L` from 24 to 28. Receptive field: unchanged (already global). Minimum input size: unchanged. Everything else: unchanged. One effect, one knob.

---

2. Wait — attention *inside* a U-Net?

Yes, and this is an important correction to a common mental model.

> **Attention is a layer. "Transformer" is an architecture.**

Attention is just an operation: `softmax(QKᵀ/√d)V`. Anyone can put it anywhere. A "Transformer" is a specific architecture that is built *entirely* out of stacked attention + MLP blocks and nothing else.

Every modern diffusion U-Net is a **hybrid**: mostly convolutions, with transformer blocks inserted at the low-resolution stages where `N = H·W` is small enough to afford `O(N²)`.

The history: DDPM (Ho et al. 2020) added a single self-attention block at the 16×16 resolution. Stable Diffusion generalised this into the `SpatialTransformer` block — a genuine little transformer, containing self-attention over spatial positions, cross-attention to text, and a feed-forward layer.

How a 2D feature map becomes a token sequence

This is the trick that lets attention live inside a conv net:

```
# incoming conv feature map
h = ...                              # (B, C, H, W)   e.g. (2, 640, 32, 32)

h = h.flatten(2)                     # (B, C, H*W)    (2, 640, 1024)
h = h.transpose(1, 2)                # (B, H*W, C)    (2, 1024, 640)
#      ^^^^ now it is EXACTLY the (B, N, d) sequence a transformer eats.
#           N = 1024 "pixel tokens", each with 640 features.

h = transformer_block(h, context)    # (B, 1024, 640)  self-attn + cross-attn + FF

h = h.transpose(1, 2)                # (B, 640, 1024)
h = h.reshape(B, C, H, W)            # (B, 640, 32, 32)  back to a conv feature map
```

Every “pixel” of the feature map is treated as one token. So a `32×32` feature map is a 1024-token sequence.

Why only at *low* resolutions

Self-attention costs `O(N²·d)`. Count the tokens:

latent resolution	`N = H·W`	`N²`	verdict
64×64	4,096	16.7 M	very expensive
32×32	1,024	1.05 M	fine
16×16	256	65 K	cheap
8×8	64	4 K	free

`64×64` costs **16× more** than `32×32`. This is precisely why SDXL *deleted* the attention blocks at the highest resolution and spent that compute on far more transformer blocks at `32×32` and `16×16` instead. It’s a budget allocation decision that someone made by running ablations.

So what actually distinguishes U-Net from DiT?

	diffusion U-Net (SD/SDXL)	DiT
trunk operation	`Conv2d`	attention + MLP
data layout	2D grid `(B,C,H,W)`	sequence `(B,N,d)`
resolutions	4–5 different ones	one
attention	at *some* resolutions only	everywhere
receptive field	grows with depth via downsampling	global from layer 1
block shapes	all different	all identical

The DiT insight was: since we already need attention for quality, and attention is the part that scales, why keep the conv scaffolding at all? Just patchify once (which is a single strided conv — one downsample) and be a pure transformer from there.

---

3. Does “bigger = better” actually hold for DiT? What knobs do I have?

What the DiT paper actually showed

The claim, stated carefully:

> Across the model family, **FID correlates tightly with training compute (Gflops per forward pass × training steps)**, and models with equal Gflops reach similar FID *even when they got those Gflops in different ways* (deeper, wider, or smaller patches). Parameter count alone predicts FID much worse than Gflops.

Two separate findings there, and both matter:

1. **Gflops is the right currency, not parameters.** DiT-XL/2 (675M params, 119 Gflops) beats DiT-XL/8 (also ~675M params, but only 5 Gflops) by an enormous margin. Same parameters, 24× the compute, dramatically better. Where you spend FLOPs matters more than how many weights you own.
2. **The *shape* of the spend is second-order.** Configurations at similar Gflops land near the same FID.

But: “no matter how I make it bigger it gets better” is too strong

Four honest caveats:

- **You must also train longer.** A bigger model at a fixed small step count can be *worse* than a small one — it hasn’t converged. The scaling law is over compute, and compute = model size × steps. DiT-XL/2 needed ~7M steps on ImageNet.
- **Extremes break.** Depth 200 at width 128 trains badly (signal propagation, attention head dim too small). Keep `d/n_heads ≈ 64` and depth/width in a sane ratio (roughly `d ≈ 64·√L` is a decent heuristic from LLM scaling work).
- **FID is not quality.** FID rewards distribution matching; it can improve while prompt-following gets worse, and it’s insensitive to text rendering. Frontier labs stopped optimising it directly.
- **Data becomes the binding constraint.** Beyond a point, more compute on the same data just memorises.

The full list of knobs

Knob	What it does	FLOPs impact	Notes / best practice
depth `L`	more sequential refinement	linear	Cheapest, safest scaling axis. Pipeline-parallel friendly.
width `d`	more features per token	**quadratic** (`d²` in every matmul)	Best FLOP utilisation (big matmuls). Tensor-parallel friendly.
patch size `p`	`N = HW/p²` tokens	attention `∝ 1/p⁴`, MLP `∝ 1/p²`	Most powerful *and* most expensive. `p=2` is standard for latents. Smaller `p` = finer detail.
`n_heads`	attention "views"	~free	Just set `d/n_heads = 64`. Not really a scaling axis.
MLP ratio	`d_ff / d`, usually 4	linear	4 is near-optimal; some models use 3 with SwiGLU (which has 3 matrices, so ratio ~2.67 keeps params equal).
input resolution	more tokens	same as patch size	Usually staged: train low-res, then high-res.
MoE experts	more params, *same* FLOPs/token	~free FLOPs, expensive memory	See §5. Decouples capacity from compute.
training steps / data	—	linear	Often the highest-return axis.

The reference DiT family, so you have real numbers:

```
name        L (depth)   d (width)   heads   params    Gflops @ 32×32 latent, p=2
DiT-S          12          384        6      33 M         6.1
DiT-B          12          768       12     130 M        23.0
DiT-L          24         1024       16     458 M        80.7
DiT-XL         28         1152       16     675 M       118.6
```

SD3-Large is `L=38, d=2432`; Flux is `L=19` double-stream + `L=38` single-stream at `d=3072`, 12B params.

Practical recipe for "make it bigger": scale `d` and `L` together (roughly `d ∝ L`), keep `head_dim=64`, keep `mlp_ratio=4`, and only reduce patch size when you specifically need finer detail. If memory-bound but not FLOP-bound, add MoE.

---

4. The four kinds of parallelism, and why uniform blocks matter

The setup: your model + activations + optimizer states don't fit on one GPU, or they fit but training would take a year. Four orthogonal ways to split the work. You typically combine them ("3D parallelism").

Baseline: DDP (Distributed Data Parallel)

```
GPU 0: full model copy, batch samples 0–7
GPU 1: full model copy, batch samples 8–15
...
after backward: all-reduce the gradients (average), each GPU applies the same update
```

Simple, near-linear speedup. **Requires the model + optimizer state to fit on one GPU.** For Adam in fp32 that's ~16 bytes/param, so a 12B model needs ~192 GB just for states. Dead on arrival at scale.

FSDP / ZeRO-3 — shard the *parameters*

```
Model has blocks B0..B27. With 4 GPUs:
  each GPU permanently stores 1/4 of every block's params, grads, and optim state.

forward, block by block:
  all-gather B0's params from all 4 GPUs → every GPU briefly has full B0
  compute B0 forward
  free the gathered params
  all-gather B1 … (and this comm OVERLAPS with B0's compute)
```

Memory drops ~`1/G`. You pay extra communication, hidden by overlapping the all-gather of block `i+1` with the compute of block `i`.

Why identical blocks matter: FSDP works on "wrapping units", and you wrap one transformer block per unit. Identical blocks → - every all-gather moves the same number of bytes → perfectly balanced, predictable - the prefetch overlap works, because comm time for block `i+1` is a known constant matched against compute time for block `i`

In a U-Net, a `1280`-channel ResBlock has ~16× the parameters of a `320`-channel one (params scale as `C²`). So the all-gather sizes vary by 16×, the comm/compute ratio swings wildly per unit, and the prefetch pipeline stalls. You *can* run FSDP on a U-Net (people do), it's just lumpy and leaves performance on the table.

Tensor parallel — shard *inside* a layer

Split individual weight matrices across GPUs. For an MLP `y = (x W1) W2`:

```
W1 is (d, 4d)  → split COLUMN-wise:  GPU_g gets W1[:, g·d : (g+1)·d]
W2 is (4d, d)  → split ROW-wise:     GPU_g gets W2[g·d : (g+1)·d, :]

GPU_g computes:  h_g = gelu(x @ W1_g)      # (B, N, d)      no comm needed
                 y_g = h_g @ W2_g          # (B, N, d)      partial sum
all-reduce(y_0 + y_1 + ... )               # exactly ONE collective per MLP
```

For attention, you shard by heads: GPU `g` owns heads `g·h/G … (g+1)·h/G`, computes those heads entirely locally, then one all-reduce after the output projection. **Two collectives per transformer block, total.** That's why TP is viable at all.

Why identical blocks matter: the sharding plan is a fixed rule — "column-split the first matrix, row-split the second, head-split attention." With 28 identical blocks you write it once. TP also needs `d` divisible by `G` and `n_heads` divisible by `G`; one width means one divisibility check.

For a U-Net you'd need a different plan per stage (different channel counts, different divisibility), and convolutions add a nastier problem: split a conv's input spatially and you need **halo exchange** (each GPU needs a 1-pixel border

from its neighbours, every layer, at every one of 5 resolutions). Channel-splitting convs avoids halos but needs an all-reduce per conv rather than per block, because a conv is *one* matrix, not a two-matrix sandwich you can fuse. Communication per FLOP is far worse.

Sequence / context parallel — shard the *tokens*

```
sequence has N = 100,000 tokens (a video). 8 GPUs:
  GPU_g holds tokens [g·12500, (g+1)·12500)   — for LayerNorm, MLP: fully local!
  attention needs every query to see every key → Ring Attention:
      pass K/V blocks around the ring, each GPU accumulates its partial softmax
      (online/flash softmax makes this exact, not approximate)
```

This is what makes long-context video generation possible at all — a 10-second 1024×576 clip is easily 100k+ tokens, and the activations alone won't fit anywhere.

Why the sequence layout matters: the whole method presumes your data *is* a 1D token sequence with a uniform per-token cost. In a DiT it literally is. A U-Net's data is a 2D grid at 5 different resolutions; "split the sequence" becomes "split the spatial grid", which reintroduces halo exchange for every conv, and you must **re-split at every resolution change** because `N` changes 4 times per forward.

Pipeline parallel — shard the *layers*

```
GPU 0: blocks  0–6      GPU 1: blocks 7–13     GPU 2: 14–20     GPU 3: 21–27
Split the batch into microbatches, flow them through like an assembly line
so all 4 GPUs are busy simultaneously (1F1B schedule).
```

Very low communication (just the activation tensor at each boundary), but needs **balanced stages** or the slowest stage throttles everything.

Why identical blocks matter: balancing 28 identical blocks over 4 GPUs is `28/4 = 7`. Done. Perfect balance, by construction.

And here is the killer for U-Nets: skip connections. In a U-Net, the encoder's stage-0 output is concatenated into the decoder's *last* block. Under pipeline parallelism that tensor must be held on GPU 0 and shipped to GPU 3 — a long-range, large (full-resolution!), cross-stage dependency. It must stay resident in memory for the entire duration of the forward pass, for every in-flight microbatch. This is close to a worst case for pipelining. The U-Net's defining architectural feature is structurally hostile to its most communication-efficient parallelism strategy.

Summary

Strategy	Splits	Comm per block	Needs uniform blocks because
DDP	batch	1 all-reduce/step	(doesn't) — but needs model to fit
FSDP / ZeRO-3	params	1–2 all-gathers	balanced comm + prefetch overlap
Tensor parallel	matrices	2 all-reduces	one sharding plan, one divisibility rule
Sequence parallel	tokens	ring K/V passes	data must be one uniform sequence
Pipeline parallel	layers	1 activation send	stage balance; and skips break it

5. Mixture-of-Experts: code, line by line, train vs inference

The idea. In a normal transformer block, every token goes through the same MLP. An MoE replaces that one MLP with `E` MLPs ("experts") and a tiny **router** that sends each token to only `k` of them (typically `k=2`).

Why: parameters and FLOPs get decoupled. With `E=8, k=2`, you have 8× the MLP parameters but each token only pays for 2 of them — so ~2/8 = 25% of the FLOPs a dense 8×-wide model would cost. Capacity is (roughly) about parameters; compute is about active parameters. MoE buys capacity cheaply.

The code

```python
import torch, torch.nn as nn, torch.nn.functional as F

class MoEFeedForward(nn.Module):
    """
    Drop-in replacement for the MLP inside a transformer block.
    Input  (B, N, d)  ->  Output (B, N, d)   plus an auxiliary load-balancing loss.
    """
    def __init__(self, d_model, d_ff, n_experts=8, k=2):
        super().__init__()
        self.d_model, self.d_ff = d_model, d_ff
        self.n_experts, self.k = n_experts, k

        # The ROUTER: one tiny linear layer that scores each token against each
        # expert. d_model -> n_experts. This is the ONLY new "decision" machinery,
        # and it is deliberately trivial (no bias, no nonlinearity) — a bigger
        # router doesn't help and makes the load-balancing dynamics worse.
        self.router = nn.Linear(d_model, n_experts, bias=False)

        # The EXPERTS. Instead of a ModuleList of n_experts separate nn.Linear
        # layers, we store all expert weights in two big 3D tensors. This lets us
        # index expert e as w_in[e] and (with grouped matmul kernels) batch all
        # experts into one kernel call. ModuleList would force E separate small
        # matmuls, which is GPU-inefficient.
        self.w_in  = nn.Parameter(torch.empty(n_experts, d_model, d_ff))  # (E, d, d_ff)
        self.w_out = nn.Parameter(torch.empty(n_experts, d_ff, d_model))  # (E, d_ff, d)
        nn.init.normal_(self.w_in,  std=d_model ** -0.5)
        nn.init.normal_(self.w_out, std=d_ff    ** -0.5)

    def forward(self, x):
        B, N, D = x.shape
        E, k = self.n_experts, self.k

        # ---- Flatten batch and sequence into one "token" axis. ---------------
        # Routing is PER TOKEN, not per sample. Two patches of the same image can
        # go to different experts. So the (B, N) structure is irrelevant here and
        # we collapse it. T = total tokens in this forward pass.
        x_flat = x.reshape(B * N, D)                       # (T, d)      T = B*N

        # ---- Route. ----------------------------------------------------------
        logits = self.router(x_flat)                       # (T, E)  raw affinities
        probs  = logits.softmax(dim=-1)                    # (T, E)  sums to 1 per token

        # Pick the top-k experts for each token.
        # topv = the gate VALUES (how much to weight each chosen expert's output)
        # topi = the expert INDICES
        topv, topi = probs.topk(k, dim=-1)                 # (T, k), (T, k)

        # Renormalise the k kept gates so they sum to 1. Without this, a token
        # whose top-2 probs were (0.3, 0.2) would have its MLP output scaled by
```

```
        # only 0.5 — i.e. the residual stream would shrink for "uncertain" tokens,
        # which destabilises training. With renormalisation every token gets a
        # convex combination, so the output magnitude is comparable to a dense MLP.
        topv = topv / topv.sum(dim=-1, keepdim=True)       # (T, k)

        # ---- Dispatch: run each expert on only its own tokens. ----------------
        out = torch.zeros_like(x_flat)                     # (T, d) accumulator

        for e in range(E):
            # Which (token, slot) pairs chose expert e?
            # topi == e is (T, k) boolean. nonzero gives us the token index and
            # which of the k slots it was, so we can look up the right gate value.
            sel = (topi == e)                              # (T, k) bool
            tok, slot = sel.nonzero(as_tuple=True)         # (Te,), (Te,)
            if tok.numel() == 0:
                continue                                   # nobody chose e this batch

            # Gather ONLY this expert's tokens. Te ≈ T*k/E on average.
            # This gather/scatter is the entire cost of MoE that a dense model
            # doesn't pay — it's memory-bandwidth-bound, not FLOP-bound.
            xe = x_flat[tok]                               # (Te, d)

            # The expert's own MLP. Note w_in[e] is (d, d_ff) — a plain matmul.
            he = F.gelu(xe @ self.w_in[e])                 # (Te, d_ff)
            ye = he @ self.w_out[e]                        # (Te, d)

            # Weight by this token's gate for this expert, then scatter-add back.
            # index_add_ handles the case where the same token appears twice
            # (once per slot) by summing — which is exactly the k-way mixture.
            g = topv[tok, slot].unsqueeze(-1)              # (Te, 1)
            out.index_add_(0, tok, ye * g)                 # (T, d) accumulated

        # ---- Auxiliary load-balancing loss (Switch Transformer form). ---------
        # THE core MoE failure mode: the router collapses. Early in training one
        # expert is slightly better, so it gets more tokens, so it trains faster,
        # so it gets even more tokens. You end up with a dense model wearing an
        # MoE costume: 1 expert used, 7 dead.
        #
        # The fix: penalise the CORRELATION between "how many tokens went to e"
        # and "how confident the router was about e".
        #   f_e = fraction of tokens dispatched to e   (a hard count — no gradient)
        #   P_e = mean router probability for e        (soft — HAS gradient)
        # L_aux = E * sum_e f_e * P_e
        # This is minimised when both are uniform (= 1/E each), giving L_aux = 1.
        # The gradient flows only through P_e, and it says: "expert e is already
        # overloaded (f_e large), so lower your probability of choosing it."
        with torch.no_grad():
            one_hot = F.one_hot(topi[:, 0], E).float()     # (T, E) top-1 assignment
            f = one_hot.mean(dim=0)                        # (E,) load fraction
        P = probs.mean(dim=0)                              # (E,) mean gate prob
        aux_loss = E * (f * P).sum()                       # scalar, ideal value 1.0
```

```
        return out.view(B, N, D), aux_loss
```

And how it's used:

```
class MoEDiTBlock(nn.Module):
    def __init__(self, d, n_heads, n_experts=8, k=2):
        super().__init__()
        self.norm1 = nn.LayerNorm(d, elementwise_affine=False)
        self.attn  = nn.MultiheadAttention(d, n_heads, batch_first=True)
        self.norm2 = nn.LayerNorm(d, elementwise_affine=False)
        self.moe   = MoEFeedForward(d, 4 * d, n_experts, k)   # <- MLP swapped for MoE
        # NOTE: attention is NOT made MoE. Attention is where tokens INTERACT;
        # making it sparse breaks information flow. The MLP is per-token, so
        # sparsifying it is safe. Essentially all MoE transformers do MLP-only.

    def forward(self, x, c):
        x = x + self.attn(self.norm1(x), self.norm1(x), self.norm1(x))[0]
        y, aux = self.moe(self.norm2(x))
        x = x + y
        return x, aux

# in the training loop:
loss = diffusion_loss + 0.01 * sum(all_aux_losses)   # aux weight ~0.01, standard
```

Train vs inference — the trade-offs

	Training	Inference
FLOPs	`k/E` of the dense-equivalent. Real win.	same win
Memory (weights)	full `E×` — all experts need gradients + Adam states. **This is the real cost.**	full `E×` resident, but no optimizer state
Parallelism	**Expert parallelism**: put experts on different GPUs, all-to-all the tokens to them and back. Two all-to-alls per MoE layer.	same
Load balance	Must be enforced with `aux_loss`, or you get collapse	Not enforced at inference — a skewed batch can hammer one expert and serialise
Determinism	fine	**A token's output can depend on the rest of the batch** if you use a capacity limit (below) — same prompt, different batch, different result
Capacity factor	Common: cap each expert at `C = cf · T·k/E` tokens (`cf` ≈1.25). Overflow tokens are **dropped** (skip the MLP, residual only). Keeps tensors static-shaped for compilers.	Usually raise `cf` or drop the cap; dropping tokens at inference is visible quality loss
Batching efficiency	large batch → every expert gets plenty of tokens → good matmul shapes	small batch (`B=1` interactive) → each expert gets a handful of tokens → tiny, inefficient matmuls. **MoE inference is much better at high batch.**

Bottom line on MoE for diffusion: it's a *memory-for-compute* trade. Use it when you're FLOP-bound but have VRAM/HBM to spare, and when you serve at high batch. It appears in large video DiTs and in some recent image models. For a from-scratch learning project: skip it, add it later — it's a drop-in swap of one module.

---

6. RoPE, NaViT packing, and the aspect-ratio / video-length problem

First: what is the actual constraint in each architecture?

U-Net. A convolution is genuinely resolution-agnostic — a `3×3` kernel slides over any size input. So a U-Net *can* run at any resolution (divisible by `2^D`). The constraints are subtler:

1. **A batch is one tensor.** `torch.Tensor` of shape `(B, C, H, W)` requires every sample to have identical `H, W`. This is a *hard* constraint and it's the practical one. You cannot put a 1024×1024 image and a 1024×576 image in the same batch. Ever. Not without padding to a common rectangle and wasting the compute.
2. **Quality doesn't transfer across scale.** SD 1.5 was trained at 512×512. Ask it for 1024×1024 and you get duplicated subjects, two heads, repeated torsos. Why? Its conv features encode "an eye is about this many pixels across"; at 2× the resolution the model sees eye-sized textures at a scale it associates with a whole face, and it composes multiple faces. This is a *learned scale prior* baked into the conv filters, and there's no mechanism to tell the model "this is the same content, just bigger."
3. **Video.** A 3D U-Net's temporal convs and temporal attention are shape-fixed per batch too — same problem in the time axis.

The workaround everyone uses is **aspect-ratio bucketing**: pre-sort your dataset into ~10 buckets (1024×1024, 1152×896, 896×1152, 1216×832, …) with roughly equal pixel counts, and make each batch come from a single bucket. It works. But: every batch is homogeneous, you need enough data per bucket, you can't mix scales within a step (which is exactly what teaches scale-invariance), and you have to re-bucket for every new resolution.

DiT. A batch is `(B, N, d)`. Different aspect ratios → different `N`. Two ways to handle heterogeneous `N` in one batch, both borrowed straight from NLP:

(a) Pad + mask. Pad every sequence to `N_max`, and pass an attention mask so padding tokens are ignored. Simple, wastes compute on padding.

(b) NaViT-style packing (Patch n' Pack). Concatenate *multiple samples'* tokens into one long sequence, and use a **block-diagonal attention mask** so tokens only attend within their own sample:

```
one packed sequence of 4096 tokens:
[ img A: 1024 tok ][ img B: 2304 tok ][ img C: 768 tok ]

attention mask (4096 × 4096), block diagonal:
     A     B     C
A [  ✓  |  ✗  |  ✗  ]
B [  ✗  |  ✓  |  ✗  ]     ✓ = allowed, ✗ = masked out
C [  ✗  |  ✗  |  ✓  ]

Zero padding waste. Three different aspect ratios in one forward pass.
FlashAttention supports this natively via cu_seqlens (varlen mode).
```

This is exactly how LLMs pack variable-length documents. It's free because the transformer's compute is *per token*, so a token doesn't care who its neighbours are — only the mask does.

Where RoPE comes in

If you handle position with a **learned absolute embedding table** `pos_emb` of shape `(N_max, d)`, you've hard-coded `N_max` and there is no principled way to evaluate at a larger `N` — row 5000 of a table with 4096 rows simply doesn't exist. Interpolating the table works okay-ish (this is what ViT does) but is a hack.

RoPE (Rotary Position Embedding) instead *rotates* the query and key vectors by an angle proportional to position, before the dot product:

```python
def rope_1d(x, pos, dim):
    # x: (B, n_heads, N, head_dim)   pos: (N,) integer positions
    # Split head_dim into pairs; rotate each pair (x_2i, x_2i+1) by angle pos * theta_i
    # with theta_i = 1 / 10000^(2i/head_dim) — same frequency ladder as sinusoidal.
    freqs = 1.0 / (10000 ** (torch.arange(0, dim, 2) / dim))   # (head_dim/2,)
    angles = pos[:, None] * freqs[None, :]                     # (N, head_dim/2)
    cos, sin = angles.cos(), angles.sin()
    x1, x2 = x[..., 0::2], x[..., 1::2]                        # even / odd channels
    return torch.stack([x1*cos - x2*sin,                       # rotation matrix
                        x1*sin + x2*cos], dim=-1).flatten(-2)

# 2D RoPE for images: split head_dim in half, apply rope_1d with the ROW index to
# one half and the COLUMN index to the other half.
def rope_2d(q, row_idx, col_idx, head_dim):
    h = head_dim // 2
    q_r = rope_1d(q[..., :h], row_idx, h)     # first half encodes vertical position
    q_c = rope_1d(q[..., h:], col_idx, h)     # second half encodes horizontal
    return torch.cat([q_r, q_c], dim=-1)
```

The magic property: because rotation is applied to *both* Q and K, the resulting dot product `q_m · k_n` depends only on `(m − n)` — the **relative** position. So:

- Nothing is indexed by absolute position → no table → no `N_max`.
- A model trained on 64×64 patch grids has learned "relative offset of 3 to the right means X". That statement is still true on a 96×64 grid. Generalisation to unseen resolutions and aspect ratios works far better.
- Video: use **3D RoPE** — split head_dim three ways for (frame, row, col). A model trained on 16-frame clips has learned relative *temporal* offsets, so 32 frames is an extrapolation of a known relation rather than a lookup into a nonexistent table.

Honest scorecard

	U-Net	DiT + RoPE
Mixed aspect ratios in one batch	**impossible** (tensor shape)	possible (pad+mask or packing)
Mixed video lengths in one batch	**impossible**	possible
Evaluate at unseen resolution	mechanically yes, quality collapses	mechanically yes, quality degrades gracefully
Standard practice in production	aspect bucketing, mandatory	aspect bucketing *still common* for efficiency, but mixing is available and used

I want to be precise here: SD3 and Flux **do** use aspect-ratio bucketing — packing isn't free (the mask and varlen kernels have overhead, and homogeneous batches are still faster). The real wins are (1) resolution generalisation from RoPE, which is why Flux handles many resolutions from one checkpoint, and (2) *the option* to mix, which matters enormously for video where clip lengths genuinely vary and padding to the max would be catastrophic.

---

7. MMDiT in code, row by row

MMDiT ("Multi-Modal Diffusion Transformer") is the SD3 / Flux block. The core idea:

> Put text tokens and image tokens in **one sequence** and let them attend jointly — but give each modality its **own weight matrices**, because a text token and an image patch are different kinds of object and shouldn't share a projection.

Sometimes called "two-stream" or "dual-stream": two sets of weights, one shared attention operation.

```python
class MMDiTBlock(nn.Module):
    """
    One MMDiT block. Takes image tokens AND text tokens, returns both, updated.

    x : (B, N, d)   image/latent tokens        N = H*W/p²,  e.g. 4096
    t : (B, M, d)   text tokens                M = 77 or 256 or 512
    c : (B, d)      the GLOBAL conditioning vector (timestep + pooled text + metadata)
    """
    def __init__(self, d, n_heads):
        super().__init__()
        self.n_heads, self.head_dim = n_heads, d // n_heads

        # ---- TWO SETS OF EVERYTHING. This is the defining feature. ----------
        # Image stream weights
        self.img_norm1 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.img_qkv   = nn.Linear(d, 3 * d)
        self.img_proj  = nn.Linear(d, d)
        self.img_norm2 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.img_mlp   = nn.Sequential(nn.Linear(d, 4*d), nn.GELU(), nn.Linear(4*d, d))
        # Its own adaLN modulation MLP: produces 6 vectors of size d
        # (shift/scale/gate for attention, shift/scale/gate for MLP)
        self.img_mod   = nn.Sequential(nn.SiLU(), nn.Linear(d, 6 * d))

        # Text stream weights — SAME SHAPES, DIFFERENT PARAMETERS.
        # Why not share? A text token's useful "query" is a semantic concept
        # ("find where the dog goes"); an image token's is a spatial/appearance
        # feature. Forcing one W_q to serve both means it must be a compromise.
        # Separate weights let each modality build its own query/key space, while
        # still landing in a SHARED d-dimensional space where dot products are
        # meaningful — that shared space is what makes joint attention work.
        self.txt_norm1 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.txt_qkv   = nn.Linear(d, 3 * d)
        self.txt_proj  = nn.Linear(d, d)
        self.txt_norm2 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.txt_mlp   = nn.Sequential(nn.Linear(d, 4*d), nn.GELU(), nn.Linear(4*d, d))
        self.txt_mod   = nn.Sequential(nn.SiLU(), nn.Linear(d, 6 * d))

        # adaLN-ZERO: zero the final modulation linear so at init every gate = 0
        # and the block is the identity. (See §14 for the full argument.)
        for mod in (self.img_mod, self.txt_mod):
            nn.init.zeros_(mod[-1].weight); nn.init.zeros_(mod[-1].bias)

    def forward(self, x, t, c):
        B, N, d = x.shape
        M = t.shape[1]

        # ---- 1. Compute modulation parameters from the global vector c. -------
        # c is (B, d). The MLP maps it to (B, 6d), which we chunk into 6 pieces of
        # (B, d). unsqueeze(1) makes them (B, 1, d) so they BROADCAST over all
        # tokens — every token of a given sample gets the same shift/scale/gate.
        # That's the whole point: c is a global signal.
```

```
        i_sh1, i_sc1, i_g1, i_sh2, i_sc2, i_g2 = self.img_mod(c).chunk(6, dim=-1)
        t_sh1, t_sc1, t_g1, t_sh2, t_sc2, t_g2 = self.txt_mod(c).chunk(6, dim=-1)
        # Note BOTH streams are modulated by the SAME c. The timestep applies to
        # the whole generation, so the text stream needs to know it too — how text
        # should be interpreted differs between "sketch the layout" (high noise)
        # and "refine the texture" (low noise).

        # ---- 2. Modulate, then project to Q,K,V — separately per stream. ------
        # AdaLN: normalise, then scale by (1+scale) and shift. The "1 +" means
        # scale=0 is the identity, which is what zero-init gives us at step 0.
        xm = self.img_norm1(x) * (1 + i_sc1.unsqueeze(1)) + i_sh1.unsqueeze(1)  # (B,N,d)
        tm = self.txt_norm1(t) * (1 + t_sc1.unsqueeze(1)) + t_sh1.unsqueeze(1)  # (B,M,d)

        xq, xk, xv = self.img_qkv(xm).chunk(3, dim=-1)   # 3× (B, N, d)
        tq, tk, tv = self.txt_qkv(tm).chunk(3, dim=-1)   # 3× (B, M, d)

        # Reshape to multi-head layout: (B, n_heads, seq, head_dim)
        def heads(z):
            return z.view(B, -1, self.n_heads, self.head_dim).transpose(1, 2)
        xq, xk, xv = map(heads, (xq, xk, xv))            # (B, H, N, hd)
        tq, tk, tv = map(heads, (tq, tk, tv))            # (B, H, M, hd)

        # (Real implementations apply 2D RoPE to xq/xk here, using each patch's
        #  (row, col). Text tokens get either no RoPE or 1D RoPE. Flux gives text
        #  position 0 and images positions 1..N in a joint index space.)

        # ---- 3. THE JOINT ATTENTION. This single line is the whole idea. ------
        # Concatenate along the SEQUENCE axis. Now we have one sequence of length
        # N+M, containing both modalities.
        q = torch.cat([tq, xq], dim=2)                   # (B, H, M+N, hd)
        k = torch.cat([tk, xk], dim=2)                   # (B, H, M+N, hd)
        v = torch.cat([tv, xv], dim=2)                   # (B, H, M+N, hd)

        # ONE attention over everything. The resulting (M+N)×(M+N) attention matrix
        # has four meaningful quadrants:
        #    txt→txt : text tokens contextualise each other (like a text encoder)
        #    txt→img : TEXT READS THE IMAGE. Impossible in cross-attention!
        #    img→txt : image patches pull in the words they need (= cross-attention)
        #    img→img : ordinary spatial self-attention
        # Cross-attention only ever gives you the img→txt quadrant. MMDiT gives all
        # four, which is what "bidirectional" means: the text representation is
        # updated by what the image currently looks like, so at high noise "a red
        # cube" can be read one way and at low noise another.
        attn = F.scaled_dot_product_attention(q, k, v)   # (B, H, M+N, hd)

        # ---- 4. Split back apart and finish each stream with ITS OWN weights. --
        attn = attn.transpose(1, 2).reshape(B, M + N, d) # (B, M+N, d)
        t_attn, x_attn = attn[:, :M], attn[:, M:]        # (B,M,d), (B,N,d)

        # Residual, gated by the adaLN-Zero gate. At init g=0 → contributes nothing.
        x = x + i_g1.unsqueeze(1) * self.img_proj(x_attn)
        t = t + t_g1.unsqueeze(1) * self.txt_proj(t_attn)
```

```
        # ---- 5. Per-stream MLP, again modulated and gated separately. ----------
        xm = self.img_norm2(x) * (1 + i_sc2.unsqueeze(1)) + i_sh2.unsqueeze(1)
        x  = x + i_g2.unsqueeze(1) * self.img_mlp(xm)
        tm = self.txt_norm2(t) * (1 + t_sc2.unsqueeze(1)) + t_sh2.unsqueeze(1)
        t  = t + t_g2.unsqueeze(1) * self.txt_mlp(tm)

        # BOTH streams are returned and flow into the next block. The text
        # representation is now a function of the image — that's the "multi-modal"
        # part. In cross-attention the text is computed once, frozen, and read
        # N times; here it is a live participant.
        return x, t
```

Contrast the three text-injection schemes side by side

```
CROSS-ATTENTION (SD1.5, SDXL, PixArt)
  text ──encode once──> C (frozen)
                        │
  image tokens ─────────┴──> Q from image, K/V from C  ──> image updated
  text is NEVER updated.  Cost: O(N·M·d).  Params: +4d²/block.

IN-CONTEXT / naive concat (early DiT ablation, works but weaker)
  cat([text, image]) ──> ONE set of weights for both ──> split
  Simple, but one W_q must serve two very different distributions.

MMDiT / JOINT (SD3, Flux)
  text ──┐                                              ┌──> text updated
         ├─> separate W's -> cat -> ONE attention -> split
  image ─┘                                              └──> image updated
  Cost: O((N+M)²·d).  Params: +8d²/block (two of everything).
```

Cost in real numbers, `N=4096, M=256, d=1536`: cross-attn adds `N·M = 1.05M` attention entries on top of the `N² = 16.8M` self-attention — about 6% overhead. MMDiT computes `(N+M)² = 18.9M` — about 13% over `N²`. So joint attention costs roughly 7% more compute than cross-attention, for a substantial quality gain (especially prompt adherence and text rendering). Flux refines this further: the first 19 blocks are dual-stream MMDiT, then the streams are merged and the final 38 blocks are single-stream (one weight set over the concatenated sequence) — the theory being that after ~19 layers the two modalities have converged into a common representation and separate weights stop paying for themselves.

---

8. Why a U-Net can *only* bolt text on from the side

This is a hard structural fact, and it's worth seeing exactly why.

The U-Net's trunk operation forbids it

The U-Net's core operation is:

```
h = nn.Conv2d(C_in, C_out, kernel_size=3, padding=1)(h)   # requires (B, C, H, W)
```

`Conv2d` needs a **2D grid**. Every element must have an `(h, w)` coordinate, because the kernel is defined by spatial offsets.

Now try to put 77 text tokens into that tensor. Ask: what is the `(h, w)` coordinate of the word “corgi”? **There isn’t one.** The question is malformed — that’s the whole reason text is a *sequence* and not a *grid*. You cannot concatenate `(B, 77, 768)` onto `(B, 320, 64, 64)` along any axis; the shapes are incompatible in kind, not just in size.

Compare a DiT: the trunk operation is

```
attn = F.scaled_dot_product_attention(q, k, v)   # requires (B, H, seq, head_dim)
```

Attention needs only a **set of vectors**. It is *permutation-equivariant* — it has no notion of position at all unless you add one. So a text token is a perfectly valid citizen of the sequence. `torch.cat([text_tokens, image_tokens], dim=1)` is type-correct and semantically meaningful. This is the entire reason MMDiT is possible in a DiT and impossible in a U-Net.

So what does a U-Net do instead? SpatialTransformer

```
class SpatialTransformer(nn.Module):
    """
    Stable Diffusion's text-conditioning block. It is a small transformer that
    temporarily converts the conv feature map into a token sequence, does
    attention, and converts back — a transformer ISLAND inside a conv net.
    """
    def __init__(self, channels, n_heads, d_context):
        super().__init__()
        self.norm    = nn.GroupNorm(32, channels)          # convs use GroupNorm,
        #                                                    not LayerNorm
        self.proj_in = nn.Conv2d(channels, channels, 1)    # 1×1 conv = per-pixel
        #                                                    linear, "enter the
        #                                                    transformer's basis"
        self.attn1   = nn.MultiheadAttention(channels, n_heads, batch_first=True)
        self.attn2   = nn.MultiheadAttention(channels, n_heads, batch_first=True)
        self.to_k    = nn.Linear(d_context, channels)      # 768 -> 640 : text lives
        self.to_v    = nn.Linear(d_context, channels)      #   in a different dim
        self.ln1 = nn.LayerNorm(channels)
        self.ln2 = nn.LayerNorm(channels)
        self.ln3 = nn.LayerNorm(channels)
        self.ff  = nn.Sequential(nn.Linear(channels, 4*channels), nn.GELU(),
                                 nn.Linear(4*channels, channels))
        self.proj_out = nn.Conv2d(channels, channels, 1)
        nn.init.zeros_(self.proj_out.weight)               # zero-init AGAIN:
        nn.init.zeros_(self.proj_out.bias)                 # this whole block is a
        #                                                    no-op at step 0

    def forward(self, h, context):
        # h       : (B, C, H, W)     conv feature map,  e.g. (2, 640, 32, 32)
        # context : (B, M, d_ctx)    text tokens,       e.g. (2, 77, 768)
        B, C, H, W = h.shape
        residual = h                                        # (B, 640, 32, 32)

        h = self.proj_in(self.norm(h))                      # (B, 640, 32, 32)

        # ---- ENTER SEQUENCE LAND ----------------------------------------------
        h = h.flatten(2).transpose(1, 2)                    # (B, 1024, 640)
        #   1024 = 32*32. Each spatial position is now one token. The 2D structure
        #   is temporarily FORGOTTEN — attention doesn't know which token is
        #   adjacent to which. SD adds no positional embedding here; it relies on
        #   the surrounding convs to have already encoded position into the
        #   features. (This is a known weakness, and part of why SD generalises
        #   poorly to new resolutions.)

        # 1. SELF-attention: pixels talk to pixels. Global spatial reasoning.
        x = self.ln1(h)
        h = h + self.attn1(x, x, x, need_weights=False)[0]  # (B, 1024, 640)

        # 2. CROSS-attention: THIS is the only channel text has.
```

```
        x = self.ln2(h)
        k = self.to_k(context)                              # (B, 77, 640)
        v = self.to_v(context)                              # (B, 77, 640)
        h = h + self.attn2(x, k, v, need_weights=False)[0]  # (B, 1024, 640)
        #   Q comes from 1024 image tokens, K/V from 77 text tokens.
        #   Attention matrix: (B, 1024, 77). Row i = "what does pixel i want from
        #   the prompt?" Each pixel gets its OWN mixture of words. This is the
        #   routing that AdaLN cannot express (see §16).
        #   NOTE: `context` is an INPUT ONLY. It is never written back. The text
        #   representation is identical in block 1 and block 40. One-way street.

        # 3. Per-token FF
        h = h + self.ff(self.ln3(h))                        # (B, 1024, 640)

        # ---- RETURN TO GRID LAND ----------------------------------------
        h = h.transpose(1, 2).reshape(B, C, H, W)           # (B, 640, 32, 32)
        return self.proj_out(h) + residual
```

The consequences, made explicit

	cross-attention (U-Net)	joint attention (DiT)
text→image information flow	yes	yes
image→text information flow	**no**	yes
text tokens contextualised by image	no	yes
text representation depth	fixed (computed once by the encoder)	grows with the trunk's depth
adding a new modality (audio, depth-as-tokens, pose)	new cross-attn module + new K/V per block, hand-plumbed	`torch.cat` it into the sequence
the trunk itself is	single-modality (conv on a grid)	modality-agnostic (attention on a set)

That last row is the strategic point. A DiT's trunk doesn't know or care what its tokens represent. Video? Just more tokens (3D RoPE). Audio? Tokens. Text? Tokens. Camera pose, depth, a reference image, an action sequence? Tokens. **The architecture generalises to new modalities for free, and the systems stack (sequence parallelism, packing, flash attention) generalises with it.** A U-Net needs a bespoke side-channel per modality, each with its own hand-tuned insertion points. This is why every "omni" generative model announced since 2024 is a transformer.

Part II — Choosing a backbone in practice

9. From scratch vs fine-tune: U-Net or DiT?

You asked three sharp questions. Let me answer them directly.

"Why not take a DiT and fine-tune it?"

Usually you should! This is the correct default whenever it's available. Fine-tuning a pretrained Flux or SD3 beats training anything from scratch, by a wide margin, for almost any natural-image task.

The question is whether it's *available* for your domain. Three things must line up:

1. The VAE latent space must match. Modern diffusion models don't operate on pixels. They operate in the latent space of a specific pretrained VAE:

```
Flux / SD3 VAE:  RGB (B,3,1024,1024)  →  latent (B,16,128,128)
SDXL VAE:        RGB (B,3,1024,1024)  →  latent (B,4,128,128)
```

The DiT has learned the statistics of *that specific* 16-channel latent space in enormous detail. Now consider:

- **3D medical volume** — `(B, 1, 256, 256, 128)`. There is no pretrained VAE for this. You'd have to train one, and then your DiT's pretrained weights are looking at a latent space they've never seen.
- **13-band multispectral satellite** — the VAE takes 3 channels. Which 3 of your 13? You've thrown away most of your signal, or you retrain the encoder.
- **Audio spectrogram** — you *can* treat it as a 1-channel image, and people do (Riffusion did exactly this with SD). But natural-image priors (edges, faces, depth-of-field) are mostly wrong for spectrograms, where the axes mean frequency and time and translation-invariance holds along one axis but not the other.
- **Materials / scientific fields** — same story, plus you may need physical constraints (mass conservation, periodic boundary conditions).

2. The conditioning signal must match. Flux conditions on T5+CLIP text embeddings. If your condition is a molecular graph, a 3D pose, a set of boundary conditions, or a tabular patient record, you're building new conditioning machinery — which is fine, but it means large parts of the model are randomly initialised anyway.

3. You must be able to afford it. Flux is 12B parameters. Full fine-tuning it needs ~8×80GB with FSDP or heavy offloading. A from-scratch 200M-parameter U-Net trains on one consumer GPU. If your budget is one 4090, "just fine-tune Flux" isn't an option, and a small model you can actually iterate on beats a big one you can't.

"Why not take a DiT and just train it from scratch?"

You can, and if you have the data and compute, you should. The problem is **sample efficiency**. A DiT has almost no inductive bias about images. It must learn from data:

- that nearby patches are correlated
- that features are translation-equivariant (a cat in the corner is still a cat)
- that spatial locality matters more than long-range for texture
- multi-scale structure

A conv net has all four **built in, for free, at initialisation.** Weight sharing across space *is* translation equivariance; a 3×3 kernel *is* locality. Those aren't learned, they're architectural constraints.

At small data, architectural constraints are worth more than flexibility, because you don't have enough samples to learn the constraint AND the task. At large data, constraints become a ceiling — the conv's locality assumption is *wrong* for global composition, and a transformer that learned the right structure from data beats one that was told a wrong-but-useful approximation. This is the same crossover as ViT vs CNN in classification: ViT loses on ImageNet-1k from scratch, wins after JFT-300M pretraining. Exactly the same phenomenon, exactly the same reason.

There are middle grounds worth knowing: - **Hybrid**: use a DiT but replace the MLP with a depthwise conv, or use windowed /neighbourhood attention in early layers. Reintroduces locality bias cheaply. - **Small patch size + convolutional patch embedding**: a 3-layer conv stem instead of a single strided conv adds local bias at the input. - **Heavy augmentation + EMA + long training** partly substitutes for data.

The decision table

Your situation	Do this	Why
Natural RGB images, style/subject/aesthetic change	**LoRA on Flux or SDXL**	10–5000 images, hours on one GPU
Natural RGB images, new domain but still photographic (product shots, medical *photos*, fashion)	**Full fine-tune SDXL, or LoRA on Flux**	pretrained features transfer
Natural RGB images, huge dataset (>10M), want frontier quality	**DiT from scratch, flow matching**	you have the data to justify it
Non-RGB modality (volumes, multispectral, spectrograms, physics fields)	**U-Net from scratch** (or train your own VAE + small DiT)	no usable checkpoint exists; conv priors help at your data scale
Spatially local task, small budget (denoise, SR, deblur)	**U-Net from scratch**	the prior matches the task; cheap
Spatially local task, want best quality, have a big GPU	**fine-tune/condition a big pretrained model**	semantic knowledge beats locality prior
One consumer GPU, learning project	**small U-Net *and* small DiT**	build both, compare — that's the point

Pros and cons, compactly

U-Net from scratch - ✅ Sample-efficient (conv priors), small, trains on one GPU, mature tooling, works on any modality/channel count, excellent at local tasks - ❌ Doesn't scale predictably, hostile to advanced parallelism, poor resolution generalisation, single-modality trunk, can't mix aspect ratios in a batch, hand-tuned architecture

DiT from scratch - ✅ Predictable scaling law, all LLM systems tooling, modality-agnostic, resolution/length generalisation with RoPE, global reasoning from layer 1, simple to specify - ❌ Data-hungry, needs a latent space (so a VAE too), more compute for the same result at small scale, `O(N²)` makes high-resolution pixel-space training infeasible

Fine-tune a pretrained DiT - ✅ Best quality per unit of your effort by a large margin; tiny data (LoRA); inherits world knowledge, text understanding, aesthetics - ❌ Requires your domain to fit the pretrained VAE + conditioning; big VRAM even for LoRA on a 12B model; licence constraints; you inherit its biases and failure modes; the base's resolution/aspect assumptions are somewhat baked in

---

10. How much data do I actually need?

Real numbers, with the caveat that everything depends on how narrow your target distribution is. Narrower = less data. These are honest working ranges, not guarantees.

Images

Goal	Data	Compute	Notes
DreamBooth / LoRA: one subject (your face, one product)	**3–20 images**	5–30 min, 1 GPU	Rank 4–32. Needs prior-preservation or regularisation images to avoid language drift.
LoRA: one style	**20–200 images**	15 min – 2 h, 1 GPU	The single highest-value technique in the ecosystem.
LoRA: a broad concept or a small domain	**500–5,000**	2–12 h, 1 GPU	Rank 32–128.
Full fine-tune SDXL (2.6B) for a domain	**10k–500k**	1–7 days, 1–8 GPUs	Watch for catastrophic forgetting; use low LR (1e-5), EMA.
Full fine-tune Flux (12B)	**10k–1M**	needs 8×A100/H100 or aggressive offload	Often LoRA at high rank is the better deal.
Continued pretraining / major domain shift	**1M–10M**	weeks, 8–64 GPUs	e.g. adapting to anime, to medical imaging, to a new resolution regime.
DiT from scratch, class-conditional, narrow domain	**~1M**	DiT-XL/2 on ImageNet: 1.28M images, 7M steps ≈ **weeks on 8×A100**	This is the *published floor* for a genuinely good class-conditional DiT.
DiT from scratch, text-to-image, competitive	**100M–2B image-text pairs**	100k+ A100-hours	SD1.5 ≈ 150k A100-hours on LAION subsets. Flux, far more. Not a solo project.
U-Net from scratch, narrow unconditional domain	**50k–200k**	hours–days, 1 GPU	FFHQ (70k faces), LSUN bedrooms — the original DDPM regime. Genuinely works.
U-Net from scratch, toy/learning	**60k (MNIST)**	minutes	What we'll build.

Note the crucial asymmetry: **fine-tuning needs 100–10,000× less data than training from scratch.** That ratio is the single most important number in this document.

So: would 10^5–10^6 work for fine-tuning a DiT? Yes, comfortably — that's squarely in "full fine-tune" territory and well above "LoRA" territory. My earlier "use a U-Net at 10^5–10^6" was specifically about the case where **no compatible pretrained checkpoint exists** (non-RGB modality). If your data is natural images, 10^5–10^6 samples means: fine-tune, don't train from scratch, and you'll get a better model than a from-scratch U-Net or a from-scratch DiT.

Video

Video is 10–100× harder on every axis. A 5-second 512×512 clip at 24fps is 120 frames — with a temporal-compressing VAE (÷4 or ÷8 in time), that's still ~15–30 latent frames × 4096 tokens = **60k–120k tokens per sample**. Sequence parallelism territory.

Goal	Data	Notes
LoRA on a video model: one motion/style	**10–100 clips**	e.g. a camera move, a specific character. Very effective.
LoRA: a broader motion domain	**500–5,000 clips**	
Full fine-tune a video DiT	**10k–500k clips**	Needs 8–64 GPUs with sequence parallelism.
From scratch video model, usable	**1M–10M clips + image pretraining**	Everyone does **joint image+video training**: images teach appearance cheaply, video teaches motion. Skipping image pretraining is a large waste.
From scratch, frontier (Sora/Veo class)	**10M–100M+ clips, curated & recaptioned**	Thousands of GPUs for months.

Video-specific practices worth knowing now: - **Progressive resolution/length curriculum.** Train on 256×256×16f, then 512×512×32f, then 1024×576×64f. Never start at the target — you'd burn your whole budget on a handful of samples. - **Image + video co-training** in the same batch (a still image is a 1-frame video). This is a major reason mixed-`N` batching (§6) matters in practice. - **Recaptioning is worth more than more data.** Sora's biggest reported lever was re-captioning the dataset with a strong VLM. Same for Flux and DALL·E 3. If you have 100k clips with bad captions, recaptioning them beats collecting 300k more. - **Condition on fps and motion magnitude** (§15) — otherwise the model averages over motion speeds and produces mush, and you have no inference-time control.

---

11. The "U-Net prior" and spatially-local tasks

What "inductive bias" / "prior" actually means, concretely

An inductive bias is an assumption the architecture makes *before seeing any data*, which constrains what functions it can represent. A conv layer makes three:

1. Locality. Output pixel `(i,j)` is computed from input pixels in a `3×3` window around `(i,j)`. Pixels far away have *literally zero* influence in one layer. Attention, by contrast, gives every position a nonzero weight on every other position by default.

2. Weight sharing → translation equivariance. The same `3×3` kernel is applied at every position. This makes an exact mathematical guarantee:

```
Conv(Shift(x)) = Shift(Conv(x))
```

Shift the input by 5 pixels, the output shifts by 5 pixels. Perfectly. A transformer with learned absolute positional embeddings does *not* satisfy this — it must learn, from examples, that a cat in the top-left and a cat in the bottom-right are the same thing. That's a huge amount of the data budget spent on something a conv gets for free by construction.

3. Scale hierarchy. The downsample ladder forces a coarse-to-fine decomposition.

Why this is worth data. Think of it as a constraint on the hypothesis space. The set of all functions from images to images is astronomically large. Convolution restricts you to the *local, translation-equivariant* subset. If the true function lives in that subset, you've deleted almost all the wrong answers for free, and your finite dataset now pins down the right one much faster. If the true function does *not* live in that subset, you've made it unreachable.

Why local tasks match this exactly

Consider 4× super-resolution. What does output pixel `(400, 300)` actually depend on?

- **Almost entirely** on input pixels around `(100, 75)` — the local texture, edges, gradients.
- **A little** on medium-range context — is this region skin, fabric, foliage?
- **Barely at all** on a pixel 900 away.

The ground-truth function for SR is *approximately local and translation-invariant* — the same "sharpen this edge" operation applies everywhere in the image. That is almost exactly the function class a conv net can express and nothing more. Perfect match. Same for denoising, deblurring, JPEG artefact removal, and to a large extent inpainting of small holes.

Contrast: "a corgi wearing a beret next to a stop sign that says HALT." Now output pixel `(400,300)` depends on: - global layout — where did the model decide to put the corgi vs the sign? - the *text token* "beret" and its binding to the head region specifically - long-range consistency — the beret must be on the head, not floating - symbolic knowledge — the letters H-A-L-T must be rendered correctly and legibly, which requires the model to relate a distant text token to a specific small region

None of this is local, and none of it is translation-equivariant (the corgi being on the left is a *different* image, not a shifted one). The conv's assumptions are now working against you: to get global information, a conv net must propagate it through many layers of downsampling, and each hop loses fidelity. Attention does it in one layer.

So does U-Net "win" at local tasks? What are people actually doing?

Careful answer, because it's genuinely split:

Yes, U-Nets/conv nets still dominate dedicated restoration. If you're shipping a 5M-parameter on-device super-resolver, or a scientific denoiser, or a real-time video upscaler, it's a conv net. Nobody sensible builds a DiT for that — you'd pay $O(N^2)$ at full resolution for reasoning you don't need.

But at the frontier, the trend is the opposite, and for an interesting reason: **real-world restoration turns out to need semantics.** If a face is blurred to 8 pixels, no local operation can recover it — the information is gone. What produces a good result is a *generative prior*: "this is a face; faces have eyes here." So the best real-world SR systems (SUPIR and its successors) are built by conditioning a large pretrained generative model, and Flux Kontext does high-quality inpainting and editing with a pure DiT. The semantic knowledge in a 12B model beats the locality prior in a 50M conv net, once you can afford it.

And the architectures are converging. Modern "transformers" for vision often reintroduce locality: windowed / neighbourhood attention, depthwise convs inside the FFN, conv patch-embedding stems. Meanwhile U-Nets adopted attention. The clean dichotomy is a teaching device, not a description of 2026 practice.

The rule to actually hold: the *prior* buys you sample efficiency at fixed compute. If you are compute- and data-limited, take the prior that matches your task. If you can afford scale, the flexible architecture wins because it learns a better prior than you could design.

---

12. Flow matching caveats: timestep sampling and the resolution shift

Recall the flow-matching setup:

```
x_t = (1−t)·x_0 + t·ε        t ∈ [0,1]     t=0 is clean data, t=1 is pure noise
target v = ε − x_0
loss = || f_θ(x_t, t, c) − v ||²
```

Why not sample `t ~ Uniform[0,1]`?

Because the difficulty of the task is wildly uneven across `t`, so uniform sampling spends most of your gradient steps on nearly-free problems.

- `t ≈ 0` (almost clean). `x_t ≈ x_0`. Predicting `v = ε − x_0` is easy: you can read `x_0` off the input almost directly, and the residual noise is small. The loss here is small and its gradients teach little.
- `t ≈ 1` (almost pure noise). `x_t ≈ ε`, so `ε` is nearly readable from the input, and the best you can do for `x_0` is the dataset mean. Also easy, also low-information.
- `t ≈ 0.5`. This is where the model must make *decisions*. There's enough signal to constrain the image but enough noise that the global structure — composition, object identity, layout — is genuinely ambiguous and must be *chosen*. All the hard modelling lives here.

So you want to concentrate sampling in the middle. **Logit-normal sampling:**

```
# SD3's choice. Sample in logit space, squash to [0,1] with sigmoid.
m, s = 0.0, 1.0                       # location, scale — m=0 centres at t=0.5
u = torch.randn(batch_size) * s + m   # u ~ N(m, s²)
t = torch.sigmoid(u)                  # (B,) in (0,1), bell-shaped, peaked at 0.5
```

The resulting density is unimodal, peaked at `t = sigmoid(m)`, with tails that still reach 0 and 1 (you must never *completely* stop training the endpoints — at sampling time you literally start at `t=1`). SD3 ablated uniform vs logit-normal vs several alternatives; logit-normal won clearly.

The resolution shift, and why it exists

Here's the phenomenon. Take the *same* noise level `t = 0.5` and apply it to a 64×64 image and to a 1024×1024 image. **The 1024 image is much easier to denoise.**

Why: natural images are spatially redundant — adjacent pixels are highly correlated. At high resolution, a given real-world feature (an eyelash) spans many pixels, all carrying nearly the same underlying information, each corrupted by *independent* noise. So you can average over the neighbourhood and the noise partially cancels: `n` independent samples of the same signal reduce noise variance by `n`. The **effective** signal-to-noise ratio of a *semantic feature* is higher at higher resolution, even though the per-pixel SNR is identical.

Empirically: destroying the global structure of a 1024×1024 image requires substantially more noise than destroying a 64×64 image. So if you train at 1024 with the same `t` distribution you used at 64, the model spends nearly all its capacity on the easy low-noise regime and never learns to construct global structure — you get locally beautiful, globally incoherent images.

The fix: shift the `t` distribution toward high noise as resolution grows.

```
def shift_t(t, s):
    """
    Monotone reparameterisation of [0,1] -> [0,1] that pushes mass toward t=1.
    s = 1  -> identity.   s > 1  -> more mass at high noise.
    Fixed points: shift(0)=0, shift(1)=1.  So the endpoints are preserved.
    """
    return s * t / (1 + (s - 1) * t)
```

Sanity check with `s = 3`: `t=0.5 → 3(0.5)/(1+2(0.5)) = 1.5/2 = 0.75`. The midpoint of your sampling distribution now corresponds to 75% noise. Exactly the intended effect.

How `s` is chosen: - SD3 uses `s ≈ 3.0` for 1024×1024 (having trained at 256 with `s = 1`), and derives it from the ratio of sequence lengths: `s ∝ √(N_new / N_old)`. The intuition is that redundancy scales with the number of tokens, and the SNR gain from averaging goes as `√n`. - Flux does **dynamic shifting**: `s` is computed at inference time from the actual sequence length, so one checkpoint handles many resolutions coherently. - Practically: if you fine-tune a flow model at a new resolution and outputs get globally incoherent or washed out, your shift is wrong. It's one of the first things to check.

Note this same fix is applied *at inference too* — the sampler's timestep schedule is shifted by the same `s`, so the sampler spends more steps in the high-noise region where the structural decisions are made.

The DDPM version of this same bug

Worth knowing because you'll see it in old code: DDPM's `ᾱ_T ≈ 0.0047 ≠ 0`, so at the highest training noise level the input still contained a faint trace of the real image (`√0.0047 ≈ 6.8%` of the signal). At sampling you start from pure noise, where that trace is absent. Train/test mismatch. Symptom: SD 1.5 cannot generate a genuinely black or genuinely white image — it always produces mid-grey-ish output, because the model learned to rely on that leaked mean brightness. The fix is "zero-terminal-SNR" rescaling of the beta schedule, plus v-prediction. **Flow matching doesn't have this bug at all**, because `x_1 = ε` exactly, by construction. One of its quietest but nicest properties.

---

13. LoRA for diffusion models

Yes — emphatically yes. LoRA is *the* dominant way diffusion models are customised. There are hundreds of thousands of public SD/SDXL/Flux LoRAs. If you work with image generation professionally, you will train LoRAs.

The mechanism

Fine-tuning updates `W → W + ΔW`. LoRA's observation: the useful `ΔW` for adaptation is **low-rank** — it lives in a small subspace. So don't store a full `d×d` matrix; factor it:

```
class LoRALinear(nn.Module):
    def __init__(self, base_linear, rank=16, alpha=16):
        super().__init__()
        self.base = base_linear                        # FROZEN pretrained weight
        for p in self.base.parameters():
            p.requires_grad = False

        d_in, d_out = base_linear.in_features, base_linear.out_features
        # Two thin matrices instead of one fat one.
        # Params: r·(d_in + d_out)  instead of  d_in·d_out
        # For d=1024, r=16: 32,768 vs 1,048,576  →  a 32× reduction.
        self.A = nn.Parameter(torch.randn(rank, d_in) * (1 / rank ** 0.5))
        self.B = nn.Parameter(torch.zeros(d_out, rank))
        #        ^^^^^^^^^^^ B is ZERO-INITIALISED.
        # So at step 0, B@A = 0, and the model is EXACTLY the pretrained model.
        # Same principle as adaLN-Zero and ControlNet's zero-convs: never inject
        # an untrained pathway at nonzero strength. If A were also zero the
        # gradient would be zero forever, so exactly one of the two is zeroed.
        self.scale = alpha / rank                      # decouples LR from rank

    def forward(self, x):
        # Two paths, summed. The base path is frozen; only A and B get gradients.
        return self.base(x) + self.scale * (x @ self.A.T @ self.B.T)
```

Where to inject it, in a diffusion model

Standard practice, in rough order of value:

1. **Cross-attention** `to_k`, `to_v` — highest value per parameter for *text* adaptation. This is where the prompt enters; changing how text maps to keys and values is how you teach a new concept or token.
2. **Cross-attention** `to_q`, `to_out`
3. **Self-attention** `q,k,v,out` — needed for style and composition changes.
4. **The FFN / MLP layers** — needed for larger domain shifts; also where most of the parameters are.
5. **Rarely:** conv layers (LoCon/LyCORIS extends LoRA to convs), the time embedding, the text encoder itself (risky — causes "language drift").

Typical settings: `rank 4–32` for a single subject or style; `rank 64–128` for a domain; `alpha = rank` or `2·rank`; LR `1e–4` (much higher than full fine-tuning's `1e–5`, because you're training a small, freshly-initialised module).

Why LoRAs are so useful in practice

- **Tiny files.** 10–200 MB vs a 5–24 GB full checkpoint. Shareable.
- **Composable.** Load two LoRAs, sum their deltas with individual weights: $W + 0.7 \cdot B_1A_1 + 0.4 \cdot B_2A_2$. A character LoRA plus a style LoRA. This is a big deal and full fine-tunes can't do it.
- **Mergeable.** $W' = W + \text{scale} \cdot BA$ at load time → **zero inference overhead**. Unlike an adapter that adds layers, a converged LoRA is free at runtime.
- **Hot-swappable at serving time**, so one base model in VRAM serves thousands of customisations.
- **Cheap to train.** Optimizer state is proportional to trainable params, so Adam's memory cost drops ~100×. This is what makes single-GPU customisation of a 12B model possible at all.

And the "no tooling transfer" point

A LoRA is a delta on *specific named weight matrices of a specific architecture*. So: - An SD1.5 LoRA does not load into SDXL (different block structure, different dimensions, different text encoders). - An SDXL LoRA does not load into Flux (U-Net vs MMDiT — the weight names don't even have counterparts). - **ControlNets are worse:** a ControlNet is a copy of the base model's encoder, so it's welded to that architecture entirely. Every new base model needs its ControlNets retrained from scratch.

That ecosystem lock-in is a genuine, underrated cost of switching architectures, and it's a large part of why SDXL remains commercially relevant despite being technically superseded.

Part III — Conditioning, re-explained from zero

14. The four ways to inject the timestep. adaLN vs adaLN-Zero

Let me restart this cleanly, because the earlier explanation moved too fast.

The problem

The model is `f(x_t, t, c)`. It must know `t`, because the correct behaviour is completely different at different noise levels: at `t=0.95` the job is "invent plausible global structure from near-noise"; at `t=0.05` the job is "sharpen existing texture, change nothing structural." **The same weights must implement both functions**, selected by `t`.

`t` starts life as a single number. First, universally, we embed it:

```
def timestep_embedding(t, dim):
    """t: (B,) float. Returns (B, dim)."""
    half = dim // 2
    # A geometric ladder of frequencies from 1 down to 1/10000.
    freqs = torch.exp(-math.log(10000) * torch.arange(half) / half)   # (half,)
    args  = t[:, None] * freqs[None, :]                               # (B, half)
    return torch.cat([args.cos(), args.sin()], dim=-1)                # (B, dim)
    # WHY sinusoids and not just the raw scalar t?
    #  1. A single scalar entering a linear layer contributes a rank-1 update —
    #     the layer can barely do anything nonlinear with it.
    #  2. Multi-frequency features make BOTH coarse ("early vs late") and fine
    #     ("t=0.500 vs t=0.505") distinctions linearly decodable. Low frequencies
    #     give smooth global position, high frequencies give local resolution.
    #  3. It's exactly the Fourier-features trick: MLPs learn high-frequency
    #     functions of a low-dimensional input very poorly; expanding to a
    #     Fourier basis fixes that.

# then always followed by an MLP, to let the network reshape that fixed basis:
t_emb = nn.Sequential(nn.Linear(dim, 4*dim), nn.SiLU(), nn.Linear(4*dim, dim))(
            timestep_embedding(t, dim))                               # (B, dim)
```

Now: how does this `(B, d)` vector reach `N` tokens? Four options.

Option 1: In-context conditioning (append as a token)

```
x = torch.cat([t_emb.unsqueeze(1), x], dim=1)   # (B, 1+N, d)  — t is token #0
# ... run all the normal transformer blocks ...
x = x[:, 1:]                                    # (B, N, d)    — drop it at the end
```

Why it's weak. Trace how `t` actually reaches image token `i`. Only through attention:

```
out_i = Σ_j  a_ij · v_j          where  Σ_j a_ij = 1     (softmax rows sum to 1)
```

So the `t` token's contribution to token `i` is `a_i,t · v_t`. Two crippling problems:

1. **It must compete for a share of a fixed budget.** The softmax row sums to 1. To give `t` an attention weight of 0.1, every image token collectively loses 0.1. Attending to `t` is *in direct competition* with attending to image content. And `t` is needed by every token at every layer, so this tax is permanent.
2. **It can only be added, never multiplied.** The output is a weighted *sum* of values. So `t` shifts the representation. It cannot **gate** — cannot say "turn feature channel 47 off entirely at high noise." Gating is what mode-switching requires.

Plus it's fragile: nothing forces the network to attend to `t` at all, and early in training it won't.

Option 2: Cross-attention to the condition

```
c_seq = torch.stack([t_emb, class_emb], dim=1)   # (B, 2, d) — a 2-token "condition"
x = x + cross_attn(q=norm(x), k=c_seq, v=c_seq)  # (B, N, d)
```

Better than option 1 — `t` now has its own dedicated channel and isn't competing with image tokens. But it's still **additive only** (weighted sum of values), still routed through a softmax, and it costs a whole extra attention module

(`4d²` params) per block to deliver a signal that has no spatial structure at all. You're paying for learned routing you don't need: there's nothing to route, since every token wants the same thing.

Option 3: adaLN (adaptive LayerNorm)

Now the multiplicative version.

```
class AdaLNBlock(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        # CRITICAL: elementwise_affine=False. A normal LayerNorm has learned
        # per-channel gain γ and bias β. Here we DELETE those learned parameters
        # and instead PREDICT them from c. That's the whole trick: the norm's
        # affine transform becomes a function of the conditioning.
        self.norm1 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.norm2 = nn.LayerNorm(d, elementwise_affine=False, eps=1e-6)
        self.attn  = nn.MultiheadAttention(d, n_heads, batch_first=True)
        self.mlp   = nn.Sequential(nn.Linear(d, 4*d), nn.GELU(), nn.Linear(4*d, d))
        # One small MLP produces 4 vectors of size d: (shift, scale) × (attn, mlp)
        self.mod = nn.Sequential(nn.SiLU(), nn.Linear(d, 4 * d))

    def forward(self, x, c):
        # x: (B, N, d)   c: (B, d)
        sh1, sc1, sh2, sc2 = self.mod(c).chunk(4, dim=-1)      # 4× (B, d)

        h = self.norm1(x) * (1 + sc1.unsqueeze(1)) + sh1.unsqueeze(1)
        #   (B,N,d)      *   (B,1,d)  broadcast    +  (B,1,d)  broadcast
        #   ^^^ MULTIPLICATIVE. sc1[k] = -1 kills channel k entirely.
        #       sc1[k] = 2 triples it. This is a GATE, per feature channel.
        #   The "1 +" means sc1 = 0 → identity, so the module starts as a no-op
        #   in the scale term and learns deviations from it. Predicting a
        #   multiplier centred on 1 is much better conditioned than predicting
        #   one centred on 0.
        x = x + self.attn(h, h, h, need_weights=False)[0]

        h = self.norm2(x) * (1 + sc2.unsqueeze(1)) + sh2.unsqueeze(1)
        x = x + self.mlp(h)
        return x
```

Why this is the right shape: - **Free routing.** `c` reaches *every* token at *full strength*, with no softmax, no competition, `O(N·d)` elementwise cost. - **Multiplicative gating.** `c` changes the *function* the block computes, not just the content it operates on. Exactly matches "select which behaviour to use." - **Symmetry match.** `c` is spatially uniform; the mechanism broadcasts uniformly. No information is lost by broadcasting because there was none to lose.

Option 4: adaLN-Zero — add a gate, initialise to zero

```
class AdaLNZeroBlock(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        # ... same as above, except:
        self.mod = nn.Sequential(nn.SiLU(), nn.Linear(d, 6 * d))   # 6, not 4
        #                                                    ^^^ two extra vectors:
        #                                          a GATE α for each residual branch
        nn.init.zeros_(self.mod[-1].weight)      # <<<< THE "ZERO"
        nn.init.zeros_(self.mod[-1].bias)        # <<<<

    def forward(self, x, c):
        sh1, sc1, g1, sh2, sc2, g2 = self.mod(c).chunk(6, dim=-1)   # 6× (B, d)

        h = self.norm1(x) * (1 + sc1.unsqueeze(1)) + sh1.unsqueeze(1)
        x = x + g1.unsqueeze(1) * self.attn(h, h, h, need_weights=False)[0]
        #       ^^^^^^^^^^^^^^^^^ the gate scales the ENTIRE residual branch

        h = self.norm2(x) * (1 + sc2.unsqueeze(1)) + sh2.unsqueeze(1)
        x = x + g2.unsqueeze(1) * self.mlp(h)
        return x
```

What "zero" buys you. At initialisation, `self.mod` has zero weight and zero bias, so *for any* `c`, its output is the zero vector. Therefore `g1 = g2 = 0`, and every block computes:

```
x = x + 0 · Attn(...) = x
x = x + 0 · MLP(...)  = x
```

The entire network is the identity function at step 0. Consequences:

- The residual stream is perfectly conditioned at the start of training — no variance explosion from 28 randomly-initialised blocks compounding. The output is `x_t` itself, passed through cleanly.
- Each block "switches itself on" only as its gradient indicates it has something useful to contribute. Effectively, the network trains shallow-to-deep.
- It also gives `sc = 0` at init, so the AdaLN scale starts at exactly the identity (thanks to the `1 +`).

The DiT paper's ablation ranked these clearly, on ImageNet 256:

```
in-context token       FID  ~ worst
cross-attention        FID  better
adaLN                  FID  better still
adaLN-Zero             FID  best  (large gap over plain adaLN)
```

And it's the same principle you see three more times across this whole field:

technique	what's zeroed	so that at step 0…
adaLN-Zero	the modulation MLP's output layer	every block is the identity
ControlNet	the `1×1` "zero convs" joining the branch to the trunk	the base model is bit-identical
LoRA	the `B` matrix	`B@A = 0`, model unchanged
SpatialTransformer `proj_out` (SD)	the output `1×1` conv	the attention block is a no-op

One general lesson: never inject an untrained pathway into a network you care about at nonzero strength. Zero it and let it fade in.

---

15. One vector or many? The canonical conditioning recipe

Direct answer to your question: **you SUM all the global (one-vector-per-image) conditions into a single `c`, and use ONE adaLN MLP per block. Sequences and spatial signals go through separate, different channels.**

So it's not "one channel" and not "a channel per signal" — it's **three channels, chosen by the shape of the signal**, with all the global signals sharing the first one.

The full recipe, as code

```
class ConditioningBundle(nn.Module):
    """
    Everything a modern text-to-image/video diffusion model conditions on,
    routed to the correct mechanism.
    """
    def __init__(self, d, d_clip_pool=1280, d_text_seq=4096, n_classes=None):
        super().__init__()
        # ---------- CHANNEL 1: the GLOBAL vector c -> adaLN --------------------
        self.t_mlp = nn.Sequential(          # timestep
            nn.Linear(256, d), nn.SiLU(), nn.Linear(d, d))
        self.pooled_proj = nn.Sequential(    # pooled/CLS text vector: 1280 -> d
            nn.Linear(d_clip_pool, d), nn.SiLU(), nn.Linear(d, d))
        self.meta_proj = nn.Sequential(      # metadata, Fourier-featurised
            nn.Linear(256 * 6, d), nn.SiLU(), nn.Linear(d, d))
        if n_classes:                        # class label (+1 row for the CFG
            self.label_emb = nn.Embedding(n_classes + 1, d)   # "null" token)

        # ---------- CHANNEL 2: the TEXT SEQUENCE -> attention ------------------
        self.text_proj = nn.Linear(d_text_seq, d)   # T5's 4096 -> model's d

        # ---------- CHANNEL 3: SPATIAL conditions -> the input tensor ----------
        # (handled at the patch-embed / conv_in, see §20)

    def forward(self, t, pooled=None, meta=None, label=None, text_seq=None):
        # ============ CHANNEL 1 ================================================
        c = self.t_mlp(timestep_embedding(t, 256))          # (B, d)  ALWAYS present

        if label is not None:
            c = c + self.label_emb(label)                   # (B, d)  SUMMED IN

        if pooled is not None:
            c = c + self.pooled_proj(pooled)                # (B, d)  SUMMED IN

        if meta is not None:
            # meta is a dict of scalars: orig_h, orig_w, crop_top, crop_left,
            # target_h, target_w  (SDXL's exact six), or for video add fps and
            # motion_bucket.
            # Each scalar gets its OWN sinusoidal embedding, then we CONCATENATE
            # (not sum) them before the MLP. Concat is safer than sum here because
            # these six numbers are semantically distinct and often correlated;
            # summing risks collisions, and concat lets the MLP learn arbitrary
            # interactions between them. Six scalars × 256 dims = 1536 -> d.
            feats = torch.cat([timestep_embedding(v, 256) for v in meta.values()],
                              dim=-1)                       # (B, 6*256)
            c = c + self.meta_proj(feats)                   # (B, d)  SUMMED IN

        # c is now ONE (B, d) vector. It goes to EVERY block's adaLN MLP.

        # ============ CHANNEL 2 ================================================
```

```
        ctx = self.text_proj(text_seq) if text_seq is not None else None  # (B,M,d)
        # ctx goes to cross-attention (U-Net) or is concatenated into the
        # sequence for joint attention (MMDiT).

        return c, ctx
```

Why summing the global signals is safe

The worry: isn't `t_emb + label_emb + pooled_emb + meta_emb` a lossy collision?

The answer is a high-dimensional geometry fact. Two independently-initialised random vectors in `ℝ^d` have expected cosine similarity `0` with standard deviation `≈ 1/√d`. At `d = 1152` that's `≈ 0.03`. So independently-learned embeddings are **very nearly mutually orthogonal**, and a sum of a handful of near-orthogonal vectors is **linearly separable**: the downstream MLP can recover each component by projecting onto its subspace. Nothing is destroyed.

This is the exact same argument that justifies `token_emb + position_emb` at the bottom of every transformer ever built, and `x + Sublayer(x)` residual streams. The residual stream is a shared additive workspace; high dimension makes it roughly a set of independent slots.

It breaks down when (a) you sum many signals (dozens), (b) the embeddings collapse onto the same direction during training, or (c) two signals are strongly correlated in the data. That last one is why SDXL **concatenates then projects** its six metadata scalars rather than summing them — `original_height` and `target_height` are highly correlated, and their interaction is what matters.

Practical rule: sum across *modalities* (time + class + pooled-text + meta), but concat-then-MLP *within* a modality (the six metadata scalars).

Your specific list, answered

Signal	Shape	Channel	Exactly how
Timestep `t`	scalar → `(B,d)`	**adaLN**	Sinusoidal embed → MLP → `c`. Non-negotiable, always adaLN, always present.
Class label	int → `(B,d)`	**adaLN**	`nn.Embedding(n_classes+1, d)`, add to `c`. The `+1` row is the **null/unconditional** token for CFG.
Image metadata (orig size, crop coords, target size, aspect ratio)	4–6 scalars	**adaLN**	Sinusoidal-embed each, **concat**, MLP, add to `c`.
Video metadata (fps, motion magnitude, camera motion)	scalars	**adaLN**	Same treatment. Gives you inference-time control over motion speed.
Pooled / CLS text vector	`(B, 768–2048)`	**adaLN**	Linear/MLP project to `d`, add to `c`. Carries global style/vibe.
Text token sequence	`(B, 77–512, 768–4096)`	**cross-attention** (U-Net, PixArt) or **joint attention** (MMDiT)	Project to `d`, then K/V for cross-attn, or concat into the sequence for MMDiT.
Image, spatially aligned (depth, edges, mask, low-res, previous video frame)	`(B, c, H, W)`	**input tensor**	VAE-encode if needed, then `torch.cat` on the channel axis; new input channels zero-initialised. Or ControlNet if the base must stay frozen.
Image, NOT aligned (identity/style reference, IP-Adapter)	`(B, M, d)` from a CLIP/DINO image encoder	**cross-attention**	A *second* cross-attn with its own K/V, summed into the text cross-attn output.
Image, as a single vector (CLIP image CLS)	`(B, d_clip)`	**adaLN**	Project, add to `c`. This is how "image variations" / unCLIP-style conditioning works.
Patch sequence from a ViT/DINO	`(B, M, d)`	depends on alignment!	Aligned to the output grid → token-concat with shared RoPE. Not aligned → cross-attention.

The two non-negotiable practices that go with this

1. Independent CFG dropout per modality. During training, randomly replace each condition with its "null" version, **independently**:

```
# per-sample masks, computed independently per modality
if training:
    drop_text  = torch.rand(B, device=dev) < 0.10     # ~10% each
    drop_pool  = torch.rand(B, device=dev) < 0.10
    drop_img   = torch.rand(B, device=dev) < 0.10
    label      = torch.where(drop_lbl, n_classes, label)      # the +1 null row
    text_seq   = torch.where(drop_text[:,None,None], null_text_emb, text_seq)
    # NOTE: independent, not a single shared mask. Independence is what lets you
    # do MULTI-condition guidance at inference: you can compute
    #   eps(x, text, img), eps(x, ∅, img), eps(x, ∅, ∅)
    # and guide each modality with its own scale. A shared dropout mask never
    # shows the model "text present, image absent", so those combinations are
    # out of distribution at inference. InstructPix2Pix relies on exactly this.
```

- `t` is **never** dropped (the model always needs the noise level).
- Metadata: usually *not* dropped, or dropped rarely — it's a nuisance parameter, not something you guide on.
- Text and image conditions: ~10% each.

2. Inference-time guidance:

```
# single-condition CFG
eps = eps_uncond + w * (eps_cond - eps_uncond)          # w ≈ 3–8

# multi-condition (e.g. text + control image), separate scales
eps = eps_none \
    + w_img  * (eps_img       - eps_none) \
    + w_text * (eps_img_text  - eps_img)
```

Two extra practices: **CFG rescaling** (after guidance, rescale the result to match the standard deviation of `eps_cond`, or high `w` oversaturates and blows out colours), and note that **many recent models are guidance-distilled** (Flux-dev, SDXL-Turbo) — guidance is baked in as an *input* to the network, so you get the CFG effect at one forward pass instead of two.

---

16. The red-cube / blue-sphere argument, slowly

You said you lost me here. Let me rebuild it from scratch, with numbers.

The setup

Prompt: **"a red cube on a blue sphere."**

You have 4096 image patch tokens. Roughly: - patches 100–400 will become the cube. They need to be **red**. - patches 2000–2400 will become the sphere. They need to be **blue**.

The question: through what mechanism does patch 200 learn "be red" while patch 2200 learns "be blue"?

Attempt A: use only a pooled text vector, through adaLN

You run the prompt through CLIP and take the pooled (CLS/EOS) output: one vector, `(1, 1280)`. Project it, add it to `c`, and feed `c` to adaLN. Now look *very carefully* at what adaLN does:

```
sh, sc = mod(c).chunk(2, -1)        # (B, d) and (B, d)  — ONE pair per image
h = norm(x) * (1 + sc.unsqueeze(1)) + sh.unsqueeze(1)
#                    ^^^^^^^^^^^^^^ shape (B, 1, d)
```

That `unsqueeze(1)` makes the shape `(B, 1, d)`, which **broadcasts** across the token axis. Concretely:

```
patch 200  receives  scale = sc,  shift = sh
patch 2200 receives  scale = sc,  shift = sh      <-- THE SAME NUMBERS
patch 4095 receives  scale = sc,  shift = sh      <-- THE SAME NUMBERS
```

Every one of the 4096 patches receives **byte-identical** modulation. There is exactly one `(scale, shift)` pair for the whole image.

So the instruction "patch 200: be red; patch 2200: be blue" **cannot be represented**. Not "is hard to represent" — cannot. The mechanism has one slot and you need 4096 different values in it. It doesn't matter if `d` is 1152 or 100,000; widening the vector doesn't create per-position slots. **It's a structural limitation, not a capacity limitation.** That's the distinction I was drawing.

(Secondary point: pooled embeddings of "a red cube on a blue sphere" and "a blue cube on a red sphere" are also *nearly identical* — same bag of words, and pooling destroys the syntactic binding. But even with a perfect pooled encoder, the broadcast problem above would still be fatal.)

Attempt B: cross-attention — and yes, this is exactly cross-attention

```
Q = image_tokens @ W_q     # (B, 4096, d)   one query per PATCH
K = text_tokens  @ W_k     # (B,    7, d)   one key per WORD
V = text_tokens  @ W_v     # (B,    7, d)

A = softmax(Q @ K.T / √d)  # (B, 4096, 7)   <-- 4096 DIFFERENT rows!
out = A @ V                # (B, 4096, d)
```

`A` is a `4096 × 7` matrix. **Row 200 is patch 200's own private distribution over the 7 words.** Row 2200 is patch 2200's own. They are computed independently, from each patch's own content. So the model can learn:

```
                "a"  "red" "cube"  "on"  "a"  "blue" "sphere"
patch  200  →   [0.01  0.45  0.40  0.02  0.01  0.05   0.06 ]   ← pulls in RED
patch 2200  →   [0.01  0.04  0.05  0.02  0.01  0.44   0.43 ]   ← pulls in BLUE
```

That's it. That's the whole answer. **Cross-attention is the mechanism that gives each output position its own private read of the condition.** AdaLN gives all positions one shared read. When you need per-position routing, you need attention. When you don't, attention is a waste.

And note the routing is *content-dependent*: which patches become the cube isn't fixed in advance — it emerges during denoising, differently for every seed. So the routing must be recomputed from the current `x_t` at every step and every layer. That's exactly what `Q = image_tokens @ W_q` gives you, and exactly what a static mechanism cannot.

The general principle behind both cases

> A mechanism can only express distinctions that its **output shape** has room for.
>
> - adaLN's output is `(B, 1, d)` broadcast → it can distinguish **images**, not positions.
> - cross-attention's output is `(B, N, d)` with `N` independent rows → it can distinguish **positions**.
>
> Ask: "does the thing I want to say differ across positions?" If yes, you need a per-position mechanism. If no, don't pay for one.

Attribute binding is the name of this failure mode — it's why weakly-conditioned models generate "a blue cube on a red sphere", or a cat with a dog's collar, or the right objects in the wrong colours. Diagnostic: if your model gets objects right but attributes swapped, look at your text conditioning path.

---

17. What SDXL, SD3, Flux actually are (a family tree)

Fair — I was using these as shorthand. Here's the map.

Stable Diffusion (SD) is a family of open-weight text-to-image *latent* diffusion models from Stability AI (originally the CompVis/LMU "Latent Diffusion Models" paper, Rombach et al. 2022). "Latent" = a VAE compresses the image 8× first, and diffusion happens in that small latent space instead of on pixels. That single idea is what made high-resolution diffusion affordable.

Model	Year	Backbone	Params (denoiser)	Objective	Text encoder(s)	Latent	Native res
DDPM	2020	U-Net + 1 attn block	35–550 M	ε-pred	none (unconditional)	pixels	32–256
ADM / guided-diffusion	2021	U-Net, more attn	550 M	ε-pred	class label	pixels	256
SD 1.4 / 1.5	2022	U-Net + cross-attn	860 M	ε-pred	CLIP ViT-L/14 (768)	4-ch, ÷8	512×512
SD 2.x	2022	U-Net + cross-attn	865 M	ε / v-pred	OpenCLIP ViT-H (1024)	4-ch, ÷8	768×768
SDXL	2023	U-Net, attn moved to low-res, 3× deeper at 32/16	**2.6 B**	ε-pred	**two**: CLIP-L + OpenCLIP-bigG (concat → 2048) **plus bigG's pooled → adaLN**	4-ch, ÷8	1024×1024
DiT	2022	**pure transformer**, adaLN-Zero	33–675 M	ε-pred	class label only	4-ch, ÷8	256/512
PixArt-α	2023	DiT + **cross-attn** + adaLN-single	600 M	ε-pred	T5-XXL (4096)	4-ch, ÷8	512–1024
SD3	2024	**MMDiT** (joint attention)	2–8 B	**flow matching**	**three**: CLIP-L + CLIP-G sequences + T5-XXL sequence → joint attn; both CLIP pooled → adaLN	**16-ch**, ÷8	1024
Flux.1	2024	MMDiT (19 dual-stream + 38 single-stream), 2D RoPE	**12 B**	flow matching (+ guidance distillation)	T5-XXL sequence + CLIP-L pooled	16-ch, ÷8	1024+, multi-aspect
Sora / Veo / Movie Gen	2024–25	DiT on **spacetime** patches, 3D RoPE	10–30 B+	flow matching	T5 / proprietary	3D VAE (space+time)	video

The three things to take from this table:

1. **The architecture flip happens between SDXL (2023) and SD3 (2024).** U-Net → MMDiT. Everything after is a transformer.
2. **The objective flips at the same time.** ε-prediction → flow matching.
3. **Text conditioning got richer and split into two channels.** SD1.5: one CLIP sequence. SDXL: two CLIP sequences **plus** a pooled vector into adaLN. SD3: three encoders, sequences into joint attention plus pooled into adaLN. And note the shift from CLIP-only to **T5** — CLIP is trained on short alt-text and is bad at long compositional prompts and at spelling; T5 is a real language model and is why SD3/Flux/PixArt can render text and follow long prompts.

Also worth knowing: the latent went from **4 channels to 16**. SD1.5's 4-channel VAE was a significant quality bottleneck (it destroys fine detail, small faces, and text). More latent channels = less lossy compression = better ceiling, at the cost of more diffusion work. If you're training from scratch today, use ≥16 channels.

---

18. CLIP pooled *and* CLIP sequence — why both

What a CLIP text encoder actually outputs

CLIP's text encoder is a transformer. Run "a red cube on a blue sphere" through it:

```
tokenize + pad to 77:   [BOS] a red cube on a blue sphere [EOS] [PAD]...[PAD]
                        ↓
transformer (12 layers)
                        ↓
last_hidden_state :  (B, 77, 768)   ← the SEQUENCE. one vector per token.
pooled_output     :  (B,     768)   ← ONE vector. Taken from the [EOS] position
                                      and then passed through a learned
                                      projection. This is the vector CLIP was
                                      CONTRASTIVELY TRAINED to align with the
                                      image embedding.
```

So yes — one model, two outputs. It's not two different text encoders (though SDXL and SD3 *do* also use multiple encoder models — see below). The pooled vector is derived from the sequence, but it's not just a mean: it's the `[EOS]` token's hidden state, which the causal transformer has used to accumulate a summary of the whole sentence, and then a projection trained specifically for image-text alignment.

What each one gives you

	pooled (`(B, 768)`)	sequence (`(B, 77, 768)`)
Contains	one global summary	per-word representations with syntax
Trained for	matching whole images (CLIP's contrastive loss)	it's just an intermediate — never directly supervised
Good at	overall style, palette, genre, "vibe", global aesthetic	which word applies to which region; counting; composition; spatial relations
Bad at	composition, binding, counting, spatial relations, spelling	providing a single global handle for style
Injected via	**adaLN** (it's one vector)	**cross-attention / joint attention** (it's a sequence)
Analogy	"make it look like *this kind of picture*"	"put *this* here and *that* there"

The pooled vector is genuinely useful and not redundant. Because it was trained contrastively against images, it lives in a space that is *already aligned with image content* — it's the closest thing CLIP has to "the image this text describes, as a single point." Injecting it into adaLN is a cheap, strong global steer: it biases every block toward a region of style space. The sequence can't do that job efficiently, because a global bias delivered through attention has to be re-extracted at every layer by every token.

Conversely, the sequence carries everything pooling destroyed. So: **both, through their shape-appropriate channels.** This is the "rule of thumb" doing its job, not an exception to it.

And SDXL/SD3 use multiple encoder *models* too

Don't conflate the two axes. There's "which outputs of an encoder do you use" (pooled vs sequence) and "how many encoders do you run."

```
SD 1.5:
   CLIP-L  ──sequence (77, 768)──> cross-attention
   (pooled unused)

SDXL:
   CLIP-L      ──sequence (77,  768)──┐
   OpenCLIP-G  ──sequence (77, 1280)──┴─concat on FEATURE axis─> (77, 2048) ─> cross-attn
   OpenCLIP-G  ──pooled       (1280)──────────────────────────────────────────> adaLN (+ metadata)

SD3:
   CLIP-L      ──sequence (77,  768) ─pad to 4096─┐
   CLIP-G      ──sequence (77, 1280) ─pad to 4096─┼─concat on TOKEN axis─> (154+77, 4096)
   T5-XXL      ──sequence (77, 4096) ─────────────┘                        ─> JOINT attention
   CLIP-L pooled (768) + CLIP-G pooled (1280) ─concat─> (2048) ────────────> adaLN

Flux:
   T5-XXL      ──sequence (512, 4096)──> joint attention
   CLIP-L      ──pooled        (768)───> adaLN
```

Why multiple encoders: they have complementary failure modes. CLIP is excellent at style and aesthetics (trained on billions of alt-text/image pairs) but its text tower is small, causal, capped at 77 tokens, and famously bad at compositional syntax and spelling. T5-XXL is a 4.7B-parameter proper language model — far better at long prompts, negation, counting, and character-level information (which is why T5-based models can render legible text in images). Using both gets you CLIP's visual-semantic grounding *and* T5's linguistic competence. The cost is that you now ship 5+ GB of frozen text encoders alongside your model; Flux drops SD3's CLIP-G and keeps only T5 sequence + CLIP-L pooled as a leaner compromise.

---

19. Why not cross-attend to a depth map? The identity-permutation argument

Let me redo this concretely.

The situation

You want to generate an image that follows a given **depth map**. The depth map is the same resolution as your latent: patch `(3, 7)` of the depth map describes exactly the region that patch `(3, 7)` of the output will occupy. The correspondence is **known, exact, and it's the identity.**

What cross-attention would have to learn

Patchify the depth map into 4096 tokens. Cross-attend:

```
A = softmax(Q_image @ K_depth.T / √d)      # (4096, 4096)
```

For this to work correctly, the model needs image token 200 to read depth token 200, image token 201 to read depth token 201, and so on. In other words, `A` must equal the **identity matrix**:

```
        depth token:  0    1    2  ...  200  201  ...
image 0            [ 1.0  0.0  0.0 ...  0.0  0.0 ]
image 1            [ 0.0  1.0  0.0 ...  0.0  0.0 ]
image 200          [ 0.0  0.0  0.0 ...  1.0  0.0 ]
```

Three things wrong with asking softmax to produce this:

1. Softmax can never output exactly one-hot. `softmax(z)_i = e^{z_i}/Σe^{z_j}`. For the diagonal entry to be `1 − δ` with 4095 competitors, you need `e^{z_diag} / (e^{z_diag} + 4095·e^{z_other}) ≈ 1`, i.e. `z_diag − z_other ≳ ln(4095/δ)`. For `δ = 0.01` that's a logit gap of about **13**. Achievable, but it means driving `QKᵀ/√d` to large magnitudes, which is numerically unpleasant (softmax saturation ⇒ vanishing gradients through the attention weights ⇒ the mechanism becomes hard to *learn* precisely at the point where it's working).

2. Every bit of residual attention mass is a blur. Suppose the model gets to 95% on the diagonal and spreads 5% over the 8 neighbours. The result is `0.95·depth[i] + 0.05·(average of neighbours)` — a slight low-pass filter of your control signal. But **spatial precision is the entire point of a depth map**. You wanted this edge exactly here. A soft, approximate identity gives you soft, approximate edges. The failure is precisely in the dimension you care about.

3. It costs `O(N²)` = 16.8M attention entries per head per layer to compute something you already knew for free.

What input-concatenation does instead

```
x     = torch.cat([noisy_latent, depth_latent], dim=1)   # (B, 4+4, 64, 64)
h     = nn.Conv2d(8, 320, 3, padding=1)(x)               # (B, 320, 64, 64)
```

Now trace the information: output feature at position `(3,7)` is computed from input channels at positions `(2..4, 6..8)` — which includes `depth[3,7]` **exactly, at full weight, with zero routing error.** The alignment is enforced by the *indexing* of the tensor, not learned. Cost: 4 extra input channels in one conv layer, ~10k parameters, no extra FLOPs worth measuring.

And why not pool the depth map into a vector for adaLN?

Because you'd compute `mean(depth_map)` (or an encoder's global pooling), getting one `(B, d)` vector, and then — per §16 — **broadcast it identically to all 4096 positions.** You'd be telling every patch the same thing about a signal whose entire content is that different positions are different. `H·W·c = 16,384` numbers compressed to `d = 1152` and then made position-invariant. All of the spatial structure — the signal itself — is gone by construction.

The general statement

> Pick the mechanism whose built-in structure already encodes the correspondence you know. If you know the correspondence exactly and it's the identity, use indexing (concatenation). Only pay for learned routing (attention) when the routing is genuinely unknown and input-dependent.

Asking a learned mechanism to reproduce a correspondence you could have hard-wired is called "burning capacity", and the model will do it *approximately*, which is the failure.

---

20. Input concat, ControlNet, token concat — a worked inpainting example

The task

Inpainting: given an image with a hole, and a mask marking the hole, generate plausible content in the hole and leave the rest alone.

The conditions are: **the known pixels** (spatially aligned) and **the mask** (spatially aligned). Both pass the alignment test, so both go into the input tensor.

Step by step, with shapes

```
# ---- Inputs ------------------------------------------------------------
image = ...        # (B, 3, 512, 512)   the original image
mask  = ...        # (B, 1, 512, 512)   1 = "regenerate this", 0 = "keep this"

# ---- 1. Build the masked image and encode both to latent space ---------
masked_image = image * (1 - mask)          # (B, 3, 512, 512)  hole is zeroed out
masked_latent = vae.encode(masked_image)   # (B, 4, 64, 64)   ÷8 by the VAE
# The mask is NOT VAE-encoded — a VAE is trained on natural images and would
# mangle a binary mask. Just downsample it geometrically to match the latent grid.
mask_latent = F.interpolate(mask, size=(64, 64), mode="nearest")   # (B, 1, 64, 64)

# ---- 2. The noisy latent we are actually denoising ---------------------
x_t = ...          # (B, 4, 64, 64)   noise added to the TARGET latent

# ---- 3. CONCATENATE on the CHANNEL axis. This is the whole mechanism. ---
model_input = torch.cat([x_t, masked_latent, mask_latent], dim=1)
#                        (B,4,..)  (B,4,..)      (B,1,..)
#             ->        (B, 9, 64, 64)
#
# Why this works with zero learning of correspondence:
#   channel 0-3 at position (i,j) = the noisy latent there
#   channel 4-7 at position (i,j) = what the image looks like there (0 if hole)
#   channel 8   at position (i,j) = is (i,j) a hole?
# All at the SAME (i,j). The tensor's indexing IS the alignment.

# ---- 4. The first conv now takes 9 channels instead of 4 ---------------
# conv_in : Conv2d(9, 320, kernel_size=3, padding=1)
h = unet.conv_in(model_input)              # (B, 320, 64, 64)
```

How you convert a pretrained 4-channel model to 9 channels

This is the practical bit, and it's a nice example of the zero-init principle again:

```
old = pretrained_unet.conv_in                    # Conv2d(4, 320, 3, padding=1)
new = nn.Conv2d(9, 320, 3, padding=1)

with torch.no_grad():
    # Copy the pretrained weights for the 4 original channels EXACTLY.
    new.weight[:, :4] = old.weight               # (320, 4, 3, 3)
    new.bias.copy_(old.bias)
    # ZERO the 5 new channels.
    new.weight[:, 4:] = 0.0                      # (320, 5, 3, 3)
    # => at step 0 the network's output is BIT-IDENTICAL to the pretrained model.
    #    The new conditioning contributes exactly nothing and fades in as it
    #    learns. Same principle as adaLN-Zero (§14) and LoRA's B matrix (§13).
    #    If you random-initialised these instead, you'd inject noise into a
    #    carefully-tuned model and destroy quality before any learning happened.
pretrained_unet.conv_in = new
```

That is literally how `stable-diffusion-inpainting` was made from `stable-diffusion-v1-5`.

Same mechanism, other tasks

Task	What you concat	Extra channels
Inpainting	masked-image latent + mask	4 + 1 = 5
Super-resolution	upsampled low-res latent	4
Depth-to-image	depth map (1 channel, resized)	1
Edge/pose-to-image	edge map or pose skeleton render	1–3
Image-to-image (InstructPix2Pix)	source-image latent	4
Video: next-frame prediction	previous frames' latents	4 per frame

ControlNet — same principle, but the base stays frozen

The problem with the above: you have to **retrain the base model.** That needs the original training data (or you'll forget things), lots of compute, and it produces a new full checkpoint per control type. If you want 10 control types you'd need 10 full 2.6B-parameter models.

ControlNet's answer:

```python
class ControlNet(nn.Module):
    def __init__(self, pretrained_unet, control_channels=3):
        super().__init__()
        # 1. FREEZE the base model entirely.
        self.base = pretrained_unet
        for p in self.base.parameters():
            p.requires_grad = False

        # 2. Make a TRAINABLE COPY of just the encoder half (down blocks + mid).
        #    Initialised from the pretrained weights — so it already knows how to
        #    process images. You are not training from scratch, you are training
        #    a copy that will learn to process the CONTROL image instead.
        self.control_encoder = copy.deepcopy(pretrained_unet.down_blocks_and_mid)

        # 3. A small conv stem to bring the control image into latent resolution.
        self.hint = nn.Sequential(
            nn.Conv2d(control_channels, 16, 3, padding=1, stride=2), nn.SiLU(),
            nn.Conv2d(16, 32, 3, padding=1, stride=2), nn.SiLU(),
            nn.Conv2d(32, 96, 3, padding=1, stride=2), nn.SiLU(),
            nn.Conv2d(96, 320, 3, padding=1))              # -> (B, 320, 64, 64)

        # 4. ZERO CONVS: one 1×1 conv per injection point, weights zeroed.
        #    A 1×1 conv is a per-position linear map — it CANNOT move information
        #    spatially. That's deliberate: the branch must inject at the position
        #    it computed, preserving alignment.
        self.zero_convs = nn.ModuleList([
            nn.Conv2d(c, c, 1) for c in [320, 320, 320, 640, 640, 640, 1280, 1280, 1280, 1280, 1280,
        1280, 1280]
        ])
        for zc in self.zero_convs:
            nn.init.zeros_(zc.weight); nn.init.zeros_(zc.bias)
            # At step 0 every injection is exactly 0 → the frozen base's output is
            # BIT-IDENTICAL to before. Zero risk of degrading the base model.

    def forward(self, x_t, t, context, control_image):
        # The control branch sees the noisy latent PLUS the control hint.
        hint = self.hint(control_image)                       # (B, 320, 64, 64)
        ctrl_h = self.base.conv_in(x_t) + hint                # (B, 320, 64, 64)

        # Run the trainable copy, collecting its per-resolution outputs.
        residuals = []
        for block, zc in zip(self.control_encoder, self.zero_convs):
            ctrl_h = block(ctrl_h, t, context)
            residuals.append(zc(ctrl_h))     # zero-conv'd, so 0 at init

        # Run the FROZEN base, adding the control residuals into its skip
        # connections at every resolution. Note residuals[i] has exactly the same
        # (C, H, W) as the skip it's added to — alignment preserved throughout.
        return self.base.forward_with_extra_residuals(x_t, t, context, residuals)
```

Why it's designed this way: - **Injection at every resolution.** Control needs to influence both coarse layout (low-res blocks) and fine edges (high-res). One injection point wouldn't do. - **1×1 zero-convs.** No spatial mixing (preserves

alignment) and no initial effect (protects the base). - **A copy of the encoder, not a fresh network.** The control image is still an image; the pretrained encoder is already an excellent image processor. Copying it is why ControlNet trains on ~50k pairs instead of millions. - **~50% of the base's parameters and compute** — the price of not touching the base. This is why ControlNet inference is roughly 1.5× the cost.

This is still **Mechanism C**: spatially aligned, position-preserving injection. Just applied at many depths and without retraining.

Trade-off: input-concat is cheaper at inference (zero overhead) and gives tighter control, but requires retraining the base and gives you one control type per checkpoint. ControlNet is modular, composable (stack several), and safe, at +50% compute. If you own the training run, concat. If you're adapting someone else's model, ControlNet.

The DiT-era form: token concatenation

A transformer has no channel axis to concatenate into — it has a sequence. So:

```
# Patchify BOTH the noisy latent and the condition image with the same patchifier.
x_tok    = patchify(x_t)              # (B, 4096, d)   the thing we're generating
cond_tok = patchify(cond_latent)      # (B, 4096, d)   the reference/control image

# Option A: concat on the FEATURE axis if perfectly aligned (equivalent to
# channel-concat — this is what a "patchify with 8 input channels" does).
tok = patchify(torch.cat([x_t, cond_latent], dim=1))     # (B, 4096, d)

# Option B: concat on the TOKEN axis — Flux Kontext's approach for editing.
tok = torch.cat([x_tok, cond_tok], dim=1)                # (B, 8192, d)
# Then give each group its own RoPE positions:
#   x_tok    gets positions (0, row, col)
#   cond_tok gets positions (1, row, col)     <- a different "image index" axis
# So attention can learn "token at (0,r,c) should look at (1,r,c)" via the
# RELATIVE positional structure RoPE provides — the alignment is carried by the
# position encoding rather than by tensor indexing.
```

Option B costs `O((2N)²) = 4×` the attention, which is real. But it buys flexibility that channel-concat can't: the reference doesn't have to be the same resolution or even the same aspect ratio, and you can supply *several* references by adding more token groups. That's why it's the form used for general image editing, where "aligned" is only approximately true.

Same principle throughout: spatial information stays spatially indexed. What changes across U-Net→DiT is only *how* you express the index — channel axis and tensor position, versus sequence position and RoPE.

Part IV

21. Learnings to remember (the compressed version)

On mechanisms

1. **Match the mechanism's shape to the signal's shape.** A mechanism can only express distinctions its output shape has room for. adaLN outputs `(B,1,d)` broadcast → distinguishes images, not positions. Attention outputs `(B,N,d)` with independent rows → distinguishes positions.

2. **Decision procedure:** does element `i` of the condition correspond to output position `i`? Yes → **input tensor** (or ControlNet / token-concat). No, and it's >1 vector → **attention**. No, and it's exactly 1 vector → **adaLN**.
3. **Modulation gates the computation; attention delivers content.** `γ ⊙ h` can switch a feature channel off; a weighted sum of values can only shift it. Use modulation for "which behaviour should this block use" (timestep, class, style), attention for "what content should this position pull in."
4. **Sum global signals into one `c`; concat-then-MLP within a modality.** `c = t_emb + class_emb + pooled_text + meta_emb` is safe because high-dimensional learned embeddings are near-orthogonal — the same reason `token_emb + pos_emb` works. But SDXL's six metadata scalars are concatenated, not summed, because they're correlated.
5. **Never inject an untrained pathway at full strength.** Zero-init the last layer and let it fade in. adaLN-Zero's gate, ControlNet's zero-convs, LoRA's `B` matrix, SD's `proj_out`, and the 5 new inpainting input channels are all the same trick.
6. **Attribute binding failures = your text is going through the wrong channel.** Right objects, swapped colours ⇒ you're relying on a pooled vector where you need per-position routing.
7. **Pooled text and sequence text are complementary, not redundant.** Pooled is contrastively aligned with images → a great global style handle for adaLN. Sequence carries syntax and binding → attention. Use both.
8. **Independent CFG dropout per modality (~10% each).** A shared mask never shows the model "text present, image absent", making that combination out-of-distribution at inference and killing multi-condition guidance.
9. **Asking a learned mechanism to reproduce a correspondence you could hard-wire is burning capacity** — and it will only manage it approximately, which shows up exactly as blur in the dimension you cared about.

On architecture

10. **Attention is a layer; Transformer is an architecture.** Diffusion U-Nets have always contained attention, at low resolutions only, because `O(N²)` makes `64×64` cost 16× what `32×32` does.
11. **Downsampling is how a conv net buys global receptive field with local operators.** Attention buys it directly. That's the whole difference.
12. **U-Net scaling is a 6-way coupled design problem; DiT scaling is two knobs.** Add a U-Net stage and you change receptive field, minimum input size, bottleneck resolution, parameter distribution, attention placement, and skip topology simultaneously.
13. **Gflops predicts FID better than parameter count.** DiT-XL/2 crushes DiT-XL/8 at identical parameters. *Where* you spend compute matters more than how many weights you own.
14. **Uniform blocks are what make advanced parallelism possible** — FSDP needs balanced all-gathers, tensor parallel needs one sharding plan, sequence parallel needs one uniform sequence, pipeline parallel needs balanced stages. And U-Net skip connections are structurally hostile to pipelining (a full-resolution tensor must live from stage 0 to stage 3).
15. **A DiT trunk is modality-agnostic because attention operates on a set, not a grid.** `Conv2d` needs an `(h,w)` for every element, and the word "corgi" doesn't have one. This is why MMDiT is impossible in a U-Net and why every "omni" model is a transformer.
16. **Cross-attention is one-way; joint attention is two-way.** In a U-Net the text representation is identical in block 1 and block 40. In MMDiT the text is updated by the image at every layer. That's worth ~7% extra compute.
17. **The inductive-bias crossover:** architectural priors buy sample efficiency at fixed compute; flexibility buys a higher ceiling at scale. Same story as CNN-vs-ViT in classification, same reason.
18. **MoE trades memory for compute.** `k/E` of the FLOPs, `E×` the weights. Apply it to the MLP only, never attention. Needs a load-balancing aux loss or the router collapses to one expert. Better at high inference batch size.
19. **RoPE makes position *relative*, which is why resolution and length generalisation work.** A learned absolute table hard-codes `N_max`; a rotation doesn't. 2D for images, 3D for video.
20. **A batch is one tensor** — this is the mundane, hard reason U-Nets can't mix aspect ratios, and why DiTs can (pad+mask or NaViT block-diagonal packing).

On training

21. **Use flow matching for anything new.** Straight paths, no schedule to tune, terminal SNR exactly zero by construction (no more "SD can't make a black image"), well-conditioned target across all `t`. Orthogonal to the backbone choice — you can use it with a U-Net.
22. **Never sample `t ~ Uniform[0,1]`.** Use logit-normal — the hard modelling all lives at mid-`t`; the endpoints are nearly free.
23. **Shift the `t` distribution toward high noise as resolution grows.** High-res images are spatially redundant, so a given noise level destroys less semantic information. `t' = s·t/(1+(s−1)·t)`, `s ≈ 3` at 1024 (`s ∝ √N`). Apply the same shift to the sampler's schedule. Globally-incoherent high-res outputs are the classic symptom of getting this wrong.
24. **Fine-tuning needs 100–10,000× less data than from-scratch.** Probably the single most actionable number here: 3–20 images for a LoRA subject, ~1M for a from-scratch class-conditional DiT, 100M+ for a from-scratch text-to-image model.
25. **"Fine-tune a pretrained DiT" is only available if your VAE latent space and conditioning match.** For 3D volumes, multispectral, or physics fields there is no checkpoint, and *that* — not data volume — is why you'd train a U-Net from scratch.
26. **LoRA is the dominant customisation method.** Inject into cross-attn `to_k` / `to_v` first (highest value per parameter for text adaptation), rank 4–32 for a subject, 64–128 for a domain. Composable, mergeable at load time so zero inference overhead, tiny files.
27. **LoRAs and ControlNets are architecture-welded.** Nothing transfers from SDXL to Flux. That ecosystem lock-in is a real cost of switching backbones and a real reason SDXL is still commercially alive.
28. **Recaptioning your dataset often beats collecting more of it.** The reported largest single lever for DALL·E 3, Sora, and Flux.
29. **For video: progressive resolution/length curriculum + joint image-video training.** Images teach appearance cheaply; video teaches motion expensively. And condition on fps and motion magnitude, or the model averages over speeds and you get mush with no inference-time control.
30. **Use ≥16 latent channels if training from scratch.** SD1.5's 4-channel VAE was a hard quality ceiling — it destroys small faces, fine detail, and text.

---

22. A prompt for another Claude terminal

Copy everything in the block below into a fresh Claude Code session in this repo.

```
I'm learning how modern diffusion models work — specifically the two backbone
families (diffusion U-Net and DiT) and, above all, HOW CONDITIONING WORKS
(timestep, class labels, text, image metadata, and image conditions).

Read `vit_mnist.py` in this repo FIRST and match its teaching style exactly.
That file is the standard: a long prose header explaining the big picture, then
numbered sections each opening with a multi-paragraph `# ===...` comment block
that explains the CONCEPT and the WHY before any code, then the code with a
shape annotation on essentially every line, e.g.:

    x = self.proj(x)        # (B, embed_dim, H/p, W/p)   e.g. (B, 256, 4, 4)
    x = x.flatten(2)        # (B, embed_dim, num_patches) e.g. (B, 256, 16)

Non-negotiable style requirements:
- Every non-trivial line gets a comment naming the shape BEFORE and AFTER, with
  a concrete example ("(B, 4, 64, 64) -> (B, 4096, 1152)").
- Every design choice gets a "why this and not the obvious alternative" comment.
  Name the alternative and say what specifically goes wrong with it.
- Every module gets a docstring stating input shapes, output shapes, and the one
  idea it implements.
- Prose explanations go in comments in the file, not just in chat. The file must
  be readable top-to-bottom as a document by someone who knows PyTorch and
  transformers but nothing about diffusion.
- Runnable and self-contained. Prefer MNIST/CIFAR-scale so it trains on Apple
  MPS in minutes. torch + torchvision + matplotlib + tqdm only.
- Add a "READING GUIDE" / "WHAT TO TRY NEXT" block at the bottom of each file,
  like train_diagnostics.py has.

Build these four files:

FILE 1 — `diffusion_01_conditioning.py`  (the toolbox; read first)
Each mechanism as a standalone, tested nn.Module with a shape trace and a
"when to use / when NOT to use" comment:
  - sinusoidal timestep embedding + MLP (explain WHY sinusoids, not raw t:
    rank-1 update, Fourier features, multi-frequency resolution)
  - LabelEmbedder with the extra "null" row for classifier-free guidance
  - FiLM vs adaLN vs adaLN-Zero, all three implemented side by side, with the
    zero-init argument spelled out and a demo that prints the network output at
    step 0 to SHOW it's the identity
  - MicroConditioning: SDXL's six scalars (original_size, crop_top_left,
    target_size) Fourier-embedded, CONCATENATED then MLP'd into the time vector.
    Explain why crop coords fix SDXL's cropped-subject problem and how you set
    them at inference for control. Include an fps / motion-bucket variant.
  - TextConditioning: pooled vector vs token sequence; what CLIP gives vs T5;
    why both are used and through DIFFERENT channels
  - CrossAttention: Q from image, K/V from text, with the (N, M) attention
    matrix explained as "each position's private read of the condition"
  - classifier_free_guidance(): the sampler-side math, independent per-modality
    dropout during training, multi-condition guidance with separate scales, and
    CFG rescaling
  - A `main()` that runs every module on dummy tensors and PRINTS the shape at
```

```
    every step, so I can read the trace.

FILE 2 — `diffusion_02_unet.py`
Full annotated diffusion U-Net in the SD/SDXL style:
  - ResBlock with time+metadata injected as scale-shift (FiLM)
  - SpatialTransformer: reshape (B,C,H,W) -> (B,HW,C), self-attn, cross-attn to
    text, FF, reshape back, zero-init proj_out. Explain the reshape as "entering
    sequence land" and why attention only lives at low resolutions (show the
    N vs N^2 token-count table).
  - down / mid / up path with skip concatenation, with a full shape trace of one
    forward pass printed as a comment block
  - image conditioning by channel-concat: a worked inpainting example going from
    4 -> 9 input channels, INCLUDING the code that surgically converts a
    pretrained 4-channel conv_in and zero-inits the new channels
  - a compact ControlNet (frozen base + trainable encoder copy + 1x1 zero convs)
    with the "why 1x1, why zero, why every resolution, why copy not fresh"
    reasoning
  - explain the receptive-field / minimum-input-size consequences of the
    downsample ladder

FILE 3 — `diffusion_03_dit.py`
  - patchify latents -> (B, N, d), and unpatchify
  - DiTBlock with adaLN-Zero: show the `mod(c).chunk(6, -1)` explicitly and
    explain each of the 6 vectors
  - final layer with zero-init
  - THEN, in the same file, three conditioning variants for text, clearly
    separated and compared:
      (a) PixArt-style: adaLN for t + cross-attention for text
      (b) MMDiT / SD3-Flux style: separate per-modality weights, ONE joint
          attention over cat([text, image]). Explain the four quadrants of the
          (M+N)x(M+N) attention matrix and why img->txt flow matters.
      (c) naive in-context (t and text as extra tokens) as the ablation that
          LOSES, with the explanation of why (softmax competes for a budget
          that sums to 1; additive only, never multiplicative)
  - 2D RoPE implemented and explained, plus a note on 3D RoPE for video and why
    relative position gives resolution generalisation
  - image conditioning in DiT-land: feature-axis concat vs token-axis concat
    (Flux Kontext style) with the cost comparison
  - a Gflops/params table for DiT-S/B/L/XL so the scaling knobs are concrete

FILE 4 — `diffusion_04_train_mnist.py`
Runnable end-to-end proof:
  - `--backbone unet|dit` and `--objective ddpm|flow` flags
  - condition on: timestep, digit class, AND a synthetic metadata signal
    (e.g. stroke thickness or rotation angle computed from the image) so I can
    SEE micro-conditioning change the output at inference
  - independent CFG dropout per modality during training
  - flow matching with logit-normal t sampling AND the resolution shift, with
    both implemented and commented; DDPM eps-prediction as the comparison
  - DDIM/Euler sampler with CFG, saving a grid of samples per epoch
  - print a table at the end comparing the two backbones and two objectives on
    the same budget
```

> Work file by file. After each file, run it, show me the printed shape traces, and give me a 5-line summary of what the file teaches before moving on. Ask me before making architectural choices that would change what I learn. Don't silently simplify — if something is too slow for MNIST scale, keep the correct code and add a comment explaining what the real version does differently.

End of document.